

PLANO MESTRE

Refatoração estrutural segura da FORJA

PRD • TDD • ROADMAP • ATLAS MERMAID • TESTES • ROLLBACK • CUTOVER

Objetivo: limpar, deduplicar, modularizar e tornar o código navegável por humanos e IAs, sem alterar comportamento jurídico, trilha de prova ou estado normativo por acidente.

Data-base	15 de julho de 2026
Versão	FORJA R1.0 — planejamento
Base Git	291fae0d57168e98ae10b0db866629b153ca9d5c
Estado	Planejado; execução não iniciada
Escopo	_FORJA_HARNESS

LIMITE DA ENTREGA — Este documento autoriza o planejamento, não a execução. Nenhum arquivo funcional foi movido, removido ou refatorado nesta etapa.

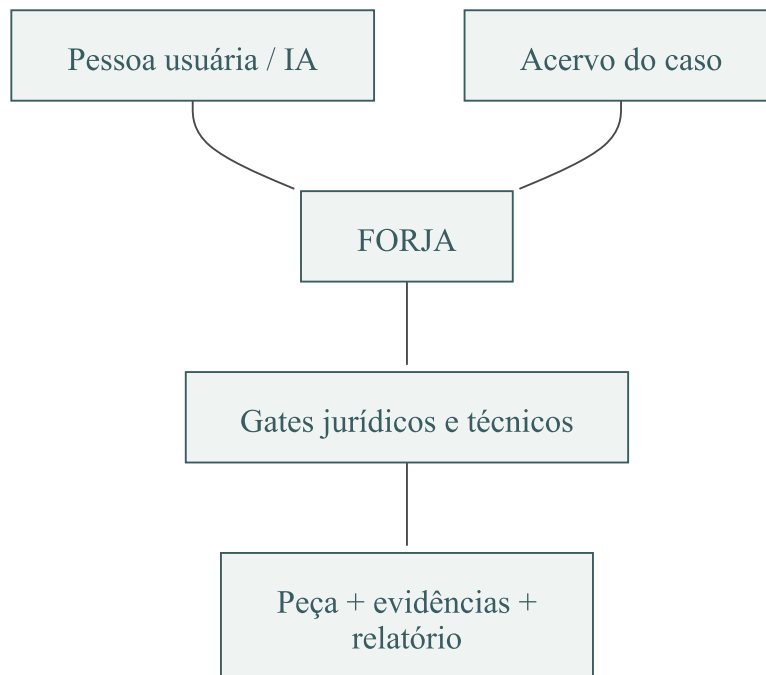
1. Síntese executiva

DECISÃO CENTRAL

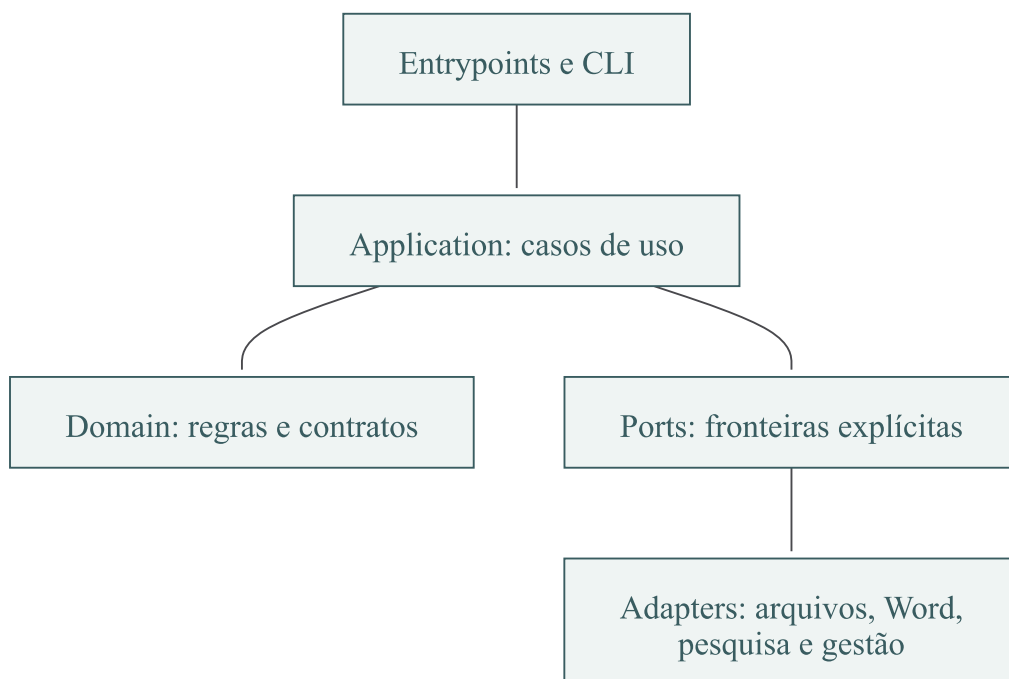
A refatoração será conduzida como migração observável, por ondas e com fachadas compatíveis. Nenhuma remoção ocorre antes de equivalência funcional, replay histórico, backup restaurável e ausência de uso do shim.

O programa FORJA R1 organiza a limpeza em dez ondas (R0–R9), 22 requisitos funcionais, 15 requisitos não funcionais, 24 sentinelas de invariantes e 18 planos executáveis. O foco imediato é construir confiança: descobrir toda a régua de testes, corrigir os quatro gates P0 e congelar um baseline reproduzível.

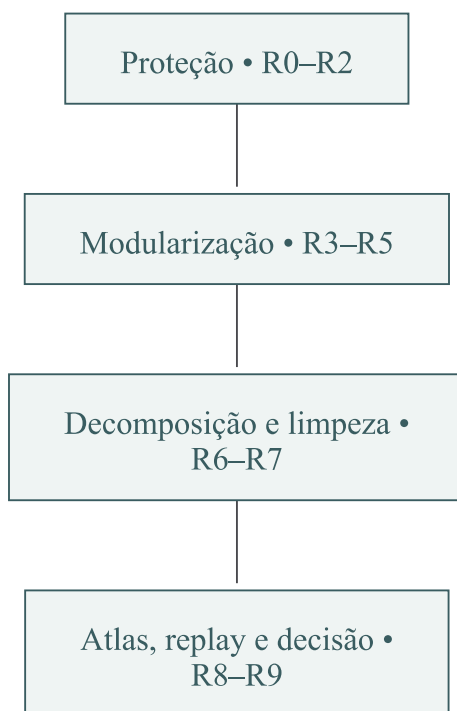
- Preservar N2 vigente, N3 em sombra e N4 em pilot_blocking; refatoração não promove especificação.
- Manter F2-A FORJA-F2A-100-v1 obrigatório em casos novos.
- Centralizar regras e catálogos sem criar um novo monólito.
- Separar domínio, aplicação, portas e adaptadores; entypoints antigos tornam-se fachadas temporárias.
- Tratar eventos, evidências, contratos e estado como patrimônio não descartável.
- Produzir atlas Mermaid vivo e manifestos legíveis por IA para reduzir mudanças às cegas.



Contexto do sistema



Camadas-alvo



Ondas do programa

Mapa de leitura

Bloco	Pergunta respondida	Fonte canônica
PRD	O que deve existir e o que não pode mudar?	01-PRD_REFATORACAO_FORJA.md
TDD	Como separar responsabilidades e testar cada mudança?	02-TDD_REFATORACAO_FORJA.md
Roadmap	Em que ordem, com quais gates e rollback?	03-ROADMAP_REFATORACAO_FORJA.md

Bloco	Pergunta respondida	Fonte canônica
Atlas	O que chama o quê e onde estão os riscos?	04-DIAGRAMAS_REFATORACAO_FORJA.md
Rastreabilidade	Qual requisito produz qual teste e evidência?	05-MATRIZ_RASTREABILIDADE.md
Cutover	Como provar equivalência e voltar atrás?	06-TESTES_ROLLBACK_E_CUTOVER.md
Planos	Qual é a unidade executável de trabalho?	18 planos em plans/P00-P16

2. Contexto e decisões travadas

Fonte canônica: 00-CONTEXT.md

FORJA R1 — Contexto travado

Coletado: 2026-07-15

Status: pronto para planejamento

Origem: pedido do Igor + pesquisa profunda do codebase

Limite do programa

Planejar e, em etapa posterior separada, executar uma refatoração estrutural segura da FORJA para reduzir duplicidade accidental, aplicar boas práticas, modularizar, organizar o código e criar documentação Mermaid viva. Este pacote não executa a refatoração.

Decisões de implementação

Preservação

- N2 permanece vigente; N3 e N4 conservam seus modos atuais.
- Gates jurídicos, event store, hashes, locks, replay, contratos históricos e QA real são preservados.
- Duplicação defensiva não será removida apenas por repetição textual.
- Toda movimentação terá teste de caracterização, fachada compatível e rollback.
- Toda exclusão futura exigirá backup verificável, prova de substituição e ensaio de restore.

Ordem

- Corrigir os quatro gates P0 e a régua de testes antes de mover fronteiras.
- Criar baseline isolado; não executar a refatoração sobre o workspace operacional sujo.
- Extrair infraestrutura neutra antes de reorganizar domínio, CLIs ou diretórios.
- Centralizar catálogos antes de gerar schemas, docs e índices.
- Remover shims somente após replay, testes reais e janela de telemetria sem uso.

Arquitetura

- Arquitetura-alvo em `src/forja: core, domain, application, ports/adapters, rendering e cli`.
- `forja_n3_common.py` vira fachada temporária, não é removido de uma vez.
- N2 entra por adaptadores; N3 permanece runtime; N4 continua camada candidata de contratos e gates.
- Integrações com Word, gestão, busca e filesystem passam por portas explícitas.
- Documentação arquitetural deve ser gerada ou validada contra AST e catálogos.

Qualidade

- TDD é obrigatório para lógica com input/output definido, validadores, estados, catálogos, seleção e promoção.
- Reorganização, configuração, documentação e migrações usam planos executivos com testes de caracterização.
- Testes reais com Word/PDF/telemetria permanecem gate de conclusão.
- Nenhum “verde” é aceito sem prova de cobertura da própria régua.

Discrição técnica do executor

- detalhes internos de classes, nomes auxiliares e sequência dentro de uma tarefa;
- escolha do gerenciador/lock compatível com o Python suportado;
- divisão interna de funções após preservação do contrato público;
- formato exato da telemetria de shims, desde que sanitizado e verificável.

Referências canônicas

Norma e governança

- `AGENTS.md` — regras invioláveis da fábrica.
- `FORJA_SPEC_MANIFEST.json` — precedência e estado N2/N3/N4.
- `FORJA_N3_CONFIG.json` — flags e modo candidato.
- `planejamento/01_PRD_FORJA.md` — produto N2 vigente.

- planejamento/05_FORJA_NIVEL_2_ANALISE_E_PLANO_CORRIGIDO.md — razões e contrato N2.
- planejamento/06_GATES_QUALIDADE_FORJA.md — gates jurídicos e visuais.

Candidatas e compatibilidade

- planejamento/08_PLANO_FORJA_N3_INTEGRIDADE_VISUAL_E_GESTAO.md — integridade N3.
- planejamento/10_PRD_FORJA_N4_RACIOCINIO_PROVA_CIENCIA.md — produto N4 candidato.
- planejamento/11_TDD_FORJA_N4_RACIOCINIO_PROVA_CIENCIA.md — contratos técnicos N4.
- planejamento/12_ROADMAP_FORJA_N4_RACIOCINIO_PROVA_CIENCIA.md — promoção N4.
- planejamento/20_F2A_EXPLORACAO_100_PERGUNTAS.md — F2-A obrigatório.

Pesquisa estrutural

- .planning/codebase/ARCHITECTURE.md — arquitetura observada.
- .planning/codebase/CHANGE_IMPACT.md — mapa de impacto.
- .planning/codebase/CONCERNS.md — riscos P0/P1/P2.
- .planning/codebase/TESTING.md — cobertura e lacunas da régua.
- .planning/codebase/STRUCTURE.md — estrutura e duplicidades físicas.

Fora do escopo

- promover N3 ou N4;
- alterar conteúdo jurídico de peças;
- novo banco, RAG, LLM-as-judge, fine-tuning ou infraestrutura paga;
- envio automático, protocolo ou assinatura;
- limpeza destrutiva de estados e evidências;
- reescrita geral do sistema.

3. PRD — requisitos e invariantes

Fonte canônica: 01-PRD_REFATORACAO_FORJA.md

PRD — FORJA R1: Refatoração Estrutural Segura

Versão: R1.0-plan

Status: planejamento; implementação não iniciada

Data: 2026-07-15

1. Visão

Transformar a FORJA em um sistema modular, reproduzível, fácil de alterar e inteligível para humanos e IAs, reduzindo duplicidades e acoplamentos sem enfraquecer gates jurídicos, rastreabilidade ou compatibilidade operacional.

A R1 não cria uma especificação jurídica nova. Ela reorganiza e fortalece a implementação das especificações existentes.

2. Problema

N2, N3 e N4 cresceram de forma aditiva. A base possui bons mecanismos de eventos, replay, locks, hashes e validação, mas concentra na raiz runtime, testes, CLIs, pilotos, geradores, saídas e scripts pontuais. Definições de artefatos, tribunais, comandos, flags e validações aparecem em múltiplos lugares.

O risco dominante não é cópia literal: é drift entre fontes de verdade e caminhos paralelos de estado, resolução e entrega. Uma IA pode editar a cópia, script histórico ou camada errada.

Antes de qualquer movimentação estrutural, quatro falhas de confiança precisam ser fechadas: régua incompleta, gate de regimento insuficientemente bloqueante, fonte jurisprudencial autocertificável e scanner de injection fail-open.

3. Estado-alvo

Estrutura conceitual: `src/forja/{core,domain,application,adapters,rendering,cli}`, `tests/{unit,contract,integration,real}`, `tools`, `experiments`, `migrations`, `archive`, `contracts`, `docs` e `var`.

4. Objetivos

- eliminar duplicidade acidental e drift;
- preservar defesa em profundidade e anti-autocertificação;
- criar fronteiras entre domínio, aplicação e integrações;
- tornar instalação e testes reproduzíveis;
- criar régua completa e auditável;
- corrigir gates fail-open antes da reorganização;
- consolidar fontes de verdade;
- reduzir custo cognitivo de mudança;
- criar atlas Mermaid vivo e mapa de impacto;
- preservar eventos, contratos, histórico e compatibilidade;
- condicionar remoção a backup, equivalência, telemetria e rollback.

5. Não objetivos

- reescrever a FORJA de uma vez;
- promover N3 ou N4;
- flexibilizar gates jurídicos;
- alterar peças, teses ou fatos;
- eliminar contratos candidatos/históricos;
- substituir Word COM;
- introduzir banco, RAG, LLM-as-judge ou infraestrutura paga;
- automatizar envio, protocolo ou assinatura;
- apagar estado, evidência ou telemetria sem política de retenção;
- executar a refatoração dentro deste pacote de planejamento.

6. Requisitos funcionais

RF-REF-001 — Baseline isolado

Registrar commit de origem, inventário, hashes, arquivos mutáveis, comandos, suíte inicial, backup e restore. Zero alteração involuntária em trabalho preexistente.

RF-REF-002 — Régua canônica completa

Descobrir e classificar 100% dos testes; falhar se `test_*.py` ficar órfão; separar unidade, contrato, integração e real; eliminar mutação global de `stdout` durante `import`; relatar omitidos e justificativa.

RF-REF-003 — Quatro gates de confiança

Regimento incorreto/incompleto/fora do caso bloqueia; fonte local permanece candidata até identidade, origem e literalidade; scanner de injection bloqueia em exceção, limite ou formato não analisado.

RF-REF-004 — Projeto reproduzível

Criar `pyproject.toml`, versão Python, grupos `word`, `visual`, `science`, `dev`, `entrypoint` forja, `lint`, `typing`, `pytest` e `lock` reproduzível.

RF-REF-005 — Fachadas compatíveis

Scripts e imports antigos delegam à implementação modular durante a janela de migração, preservando interface, `exit code` e `outputs` relevantes.

RF-REF-006 — Catálogos canônicos

Criar fontes únicas e tipadas para fases/estados, tribunais/CNJ, artefatos, severidades, comandos, gates e invalidação.

RF-REF-007 — Geração verificável

Derivar schemas, contratos, flags, índices e documentação quando aplicável. Um modo `--check` detecta drift sem escrever.

RF-REF-008 — Preservação do event store

Manter `append-only`, idempotência, revisão otimista, replay determinístico, escrita atômica, compatibilidade e rejeição de regressão silenciosa.

RF-REF-009 — Promoção transacional

Revalidar `contextHash`, `contractHash`, artefatos e gates imediatamente antes da promoção. Falha não deixa visão canônica parcial.

RF-REF-010 — Resolvedor canônico de caso

Zero, um ou múltiplos matches têm resultado explícito. Nenhum runtime escolhe silenciosamente o primeiro caso.

RF-REF-011 — Entrega por identidade e hash

Selecionar artefato por ID/hash; `glob` fica somente em adaptador legado com aviso de ambiguidade; `retry` é idempotente.

RF-REF-012 — Outbox de gestão

Desacoplar sincronização da transação principal por outbox persistente e idempotente. Falha do painel não perde evento do caso.

RF-REF-013 — Porta de renderização

Isolar `template`, `Word COM`, `SVG→EMF`, `PDF` e `QA` por contratos explícitos, distinguindo capacidade ausente, falha de `render` e `QA` bloqueado.

RF-REF-014 — Decomposição de hotspots

Dividir `forja_visual.compor`, `forja_delivery.main`, `forja_render_docx.render`, `forja_pso_pet.validate_plan`, `forja_n4_m6_cycles.run`, `validate_adversarial_audit` e `forja_n4_validate.validate_case` após testes de caracterização.

RF-REF-015 — Separação física

Separar runtime, testes, ferramentas, experimentos, migrações, histórico e `outputs`. Imports/buscas do runtime não percorrem `var`, `archive` ou projeto estrangeiro.

RF-REF-016 — Sanitização por manifesto

Classificar cada item como canônico, wrapper, experimento, migração, output, duplicidade acidental, defesa em profundidade ou histórico protegido. Exclusão exige backup e restore.

RF-REF-017 — CLI consolidada

Criar entripoint único e subcomandos; scripts antigos viram wrappers; parsing não contém regra jurídica.

RF-REF-018 — Taxonomia de erros

Distinguir configuração inválida, capacidade indisponível, entrada não verificada, bloqueio jurídico, falha transitória, conflito de revisão, ambiguidade e erro interno. Biblioteca não chama `SystemExit`.

RF-REF-019 — Atlas Mermaid vivo

Manter diagramas de contexto, N2/N3/N4, F0–F10/F2-A, imports, eventos, attempts, render, fontes, entrega, catálogos, testes, invalidação, ownership, impacto e rollback. Todos renderizam e os estruturais são verificados contra código.

RF-REF-020 — Navegação para IA

Cada domínio documenta responsabilidade, entrada, saída, fonte de verdade, invariantes, consumidores, testes, riscos e rollback. A matriz “alterar X → verificar Y” cobre as fontes canônicas.

RF-REF-021 — Remoção observável de shims

Wrappers emitem telemetria sanitizada. Remoção exige ausência de referência, equivalência, janela sem uso e rollback.

RF-REF-022 — Preservação F2-A

Manter 100 perguntas, 10 óticas, IDs, `supportIds`, lacunas bloqueadas, duas soluções, handoff F3–F7, compatibilidade histórica e bloqueio por questão material.

7. Requisitos não funcionais

ID	Requisito	Prova
RNF-001	segurança jurídica	zero gate degradado sem decisão normativa
RNF-002	compatibilidade	100% do corpus selecionado legível e replay equivalente
RNF-003	auditabilidade	todo movimento registra origem, destino, hash, teste e rollback
RNF-004	fail-closed	exceção/não analisado nunca vira ok
RNF-005	determinismo	duas execuções limpas produzem equivalência declarada
RNF-006	modularidade	teste de imports impede domínio → infraestrutura
RNF-007	manutenibilidade	APIs públicas tipadas e orçamento de complexidade reduzido
RNF-008	reprodutibilidade	instalação limpa executa suíte não-real
RNF-009	desempenho	sem regressão material não justificada
RNF-010	portabilidade controlada	núcleo roda sem Word; entrega real declara Windows
RNF-011	observabilidade	IDs, hashes, duração, resultado e erro sanitizado
RNF-012	privacidade	scanner de segredos e proveniência limpo
RNF-013	legibilidade	Mermaid sem erro e texto ≥8 pt no PDF
RNF-014	recuperabilidade	rollback ensaiado por marco
RNF-015	limite normativo	manifest não promove spec por implicação

8. Sentinelas de invariantes

Os itens INV-* abaixo **não criam norma nova**. São sentinelas técnicas para detectar regressão contra as fontes canônicas. Em conflito ou drift, prevalecem `../AGENTS.md`, `FORJA_SPEC_MANIFEST.json`, `FORJA_N3_CONFIG.json` e os protocolos próprios citados pelo AGENTS.

ID	Invariante preservado
INV-01	N2 permanece vigente até promoção formal.
INV-02	Painel resume; comando orienta; anexo/fonte prova.
INV-03	Regimento integral, correto e atualizado antes da redação.
INV-04	<code>_LEIS_GERAIS</code> , Estatuto da OAB e LOMAN permanecem no fluxo.
INV-05	Fonte não oficial só descobre; uso final exige fonte validada.
INV-06	Fato, declaração, inferência, hipótese e não verificado permanecem separados.
INV-07	Proveniência interna nunca entra na peça protocolável.
INV-08	Processo volumoso exige cronologia e identidade de atos.
INV-09	F2-A bloqueia saída externa se houver questão material pendente.
INV-10	Helena e Cícero permanecem gates obrigatórios.
INV-11	Auditoria adversarial permanece em peças responsivas.
INV-12	Documento protocolável nasce do template ou peça anterior.
INV-13	PDF final continua por Word COM.
INV-14	Toda regeneração exige QA de todas as páginas.
INV-15	Metadados finais são institucionais, nunca <code>python-docx/IA</code> .
INV-16	<code>pronta_para_revisao</code> não é cumprida.
INV-17	Entrega ao escritório exige evidência; protocolo é fronteira distinta.
INV-18	Sem envio, protocolo ou assinatura automáticos.
INV-19	Bloquear por falta de fonte/prova é sucesso de segurança.
INV-20	Eventos, invalidados e contratos históricos não são apagados.
INV-21	Recálculo independente de gates permanece.
INV-22	“Não localizado” não vira “inexistente”.
INV-23	Má-fé exige lastro e aprovação humana/Cícero.
INV-24	Prescrição administrativa mantém matriz específica do AGENTS.

Proveniência normativa das sentinelas

Sentinelas	Fonte canônica	Seção/data de referência	Controle de drift
INV-01, INV-20, INV-21	<code>FORJA_SPEC_MANIFEST.json</code> ; <code>FORJA_N3_CONFIG.json</code>	snapshot do baseline R0	hash bruto + diff semântico
INV-02, INV-16–INV-19	<code>../AGENTS.md</code>	§8 Hermes/gestão viva; atualizado em 14/07/2026	hash/data + testes de gestão/entrega
INV-03–INV-05	<code>../AGENTS.md</code>	§§1, 2, 7; atualizado em 14/07/2026	hash/data + gates de fonte/regimento
INV-06–INV-08, INV-22–INV-24	<code>../AGENTS.md</code>	§§7-A, 7-B, 7-D; atualizado em 14/07/2026	hash/data + corpus adversarial
INV-09	<code>../AGENTS.md</code> ; <code>templates/F2A_EXPLORACAO_100_PERGUNTAS.md</code>	§7-C; contrato <code>FORJA-F2A-100-v1</code>	hash do template/validador
INV-10, INV-11	<code>FORJA_SPEC_MANIFEST.json</code> ; planos N3/N4	snapshot do baseline R0	hash + replay
INV-12–INV-15	<code>../AGENTS.md</code> ; <code>../_FERRAMENTAS/PADRAO_WORD_MEDINA_OSORIO.md</code>	§§3, 3-A, 4, 6	hash/data + QA real

R0 registra os hashes vigentes. Alteração posterior de fonte canônica invalida a sentinela derivada até revisão explícita; o PRD nunca sobrepõe a fonte.

9. Critérios globais de aceite

1. Quatro P0 corrigidos com testes negativos.
2. Todos os testes descobertos e classificados.
3. Unidade, contrato e integração integralmente verdes.
4. Word/PDF/telemetria verdes em casos-modelo.
5. Replay histórico equivalente.
6. N2/N3/N4 conservam estado normativo.
7. F2-A conserva contrato e compatibilidade.
8. Zero import proibido entre camadas.
9. Catálogos canônicos substituem definições divergentes.
10. Geradores têm --check determinístico.
11. Caso e entrega são resolvidos por identidade/hash.
12. Falha de promoção não deixa parcialidade silenciosa.
13. Gestão opera por outbox idempotente.
14. Wrappers permanecem enquanto houver uso.
15. Mermaid corresponde ao código e renderiza.
16. Ambiente nasce de manifesto/lock.
17. Outputs ficam separados do fonte.
18. Nenhum histórico protegido é perdido.
19. Backup e rollback são ensaiados.
20. Artefatos jurídicos não sofrem regressão factual, jurídica ou visual.
21. Auditoria de segredo/proveniência fica limpa.
22. Documentação informa onde alterar, impacto e testes.
23. Uma IA nova navega sem editar cópia, output ou experimento.
24. Todo shim removido possui evidência individual de não uso.

4. TDD — desenho técnico e testes

Fonte canônica: 02-TDD_REFATORACAO_FORJA.md

TDD — FORJA R1: Refatoração Estrutural Segura

Versão: R1.0-plan

Status: desenho técnico; execução não iniciada

Princípio: uma mudança de comportamento por plano TDD; movimentações puras exigem caracterização e equivalência

1. Decisão arquitetural

R1 é um programa separado da promoção N3/N4. N2 continua vigente, N3 permanece em sombra e N4 em `pilot_blocking`. A refatoração usa `strangler pattern`: scripts e imports atuais chamam fachadas, que passam a delegar gradualmente a módulos novos.

2. Estrutura-alvo

```
src/forja/
├── core/
│   ├── errors.py
│   ├── result.py
│   ├── json_io.py
│   ├── hashing.py
│   ├── locking.py
│   ├── paths.py
│   ├── ids.py
│   └── time.py
├── domain/
│   ├── phases.py
│   ├── tribunals.py
│   ├── artifacts.py
│   ├── severity.py
│   ├── release_policy.py
│   ├── transitions.py
│   └── gates/
├── application/
│   ├── prepare_attempt.py
│   ├── promote_attempt.py
│   ├── validate_case.py
│   ├── build_package.py
│   ├── close_cycle.py
│   ├── reconcile_case.py
│   └── sync_management.py
├── ports/
│   ├── case_repository.py
│   ├── event_store.py
│   ├── renderer.py
│   ├── legal_sources.py
│   ├── management.py
│   └── delivery.py
├── adapters/
│   ├── filesystem/
│   ├── word_medina/
│   ├── management_sidecar/
│   ├── legal_search/
│   └── legacy_n2/
├── rendering/
└── cli/
```

3. Interfaces de fronteira

```
class CaseResolver(Protocol):
    def resolve(self, key: str | Path) -> Path: ...

class EventStore(Protocol):
    def read(self, case_id: str) -> tuple[Event, ...]: ...
    def append(self, event: Event, expected_revision: int) -> AppendResult: ...
    def replay(self, case_id: str) -> CaseState: ...
```

```

class ArtifactRepository(Protocol):
    def stage(self, attempt: AttemptId, artifact: ArtifactCandidate) -> ArtifactDigest: ...
    def promote_atomically(self, plan: PromotionPlan) -> PromotionReceipt: ...

class Gate(Protocol):
    name: str
    def evaluate(self, context: GateContext) -> GateDecision: ...

class LegalSourceVerifier(Protocol):
    def identify(self, candidate: SourceCandidate) -> SourceIdentityResult: ...
    def verify_content(self, identity: SourceIdentity, claim: Claim) -> SourceVerification: ...

class DocumentRenderer(Protocol):
    def capabilities(self) -> RenderCapabilities: ...
    def render(self, request: RenderRequest) -> RenderReceipt: ...
    def inspect_all_pages(self, receipt: RenderReceipt) -> VisualQaReceipt: ...

class ManagementOutbox(Protocol):
    def enqueue(self, event: ManagementEvent) -> OutboxReceipt: ...
    def flush(self) -> SyncReport: ...

class DeliveryPort(Protocol):
    def prepare(self, package: AuditedPackage) -> DeliveryDraft: ...
    def confirm(self, draft: DeliveryDraft, evidence: DeliveryEvidence) -> DeliveryReceipt: ...

```

Essas assinaturas são contratos de planejamento. Nomes auxiliares podem ser ajustados na implementação, mas a separação de responsabilidade e os comportamentos testáveis são obrigatórios.

4. ADRs propostos

ADR	Decisão
ADR-001	Arquitetura strangler com fachadas até remoção comprovada.
ADR-002	Eventos são fonte canônica N3; snapshots são materializações.
ADR-003	ArtifactCatalog tipado gera/verifica schemas, flags, métricas e docs.
ADR-004	Domínio não importa Word, gestão, rede ou filesystem concreto.
ADR-005	Gate crítico falha fechado em erro, ausência ou item não examinado.
ADR-006	Seleção por <code>artifactId</code> + hash; glob legado rejeita ambiguidade.
ADR-007	Gestão usa outbox file-first idempotente fora da transação do caso.
ADR-008	Regras determinísticas não são autocertificadas por saída inteligente.
ADR-009	Estado, cache, telemetria e renders são dados mutáveis com retenção própria.
ADR-010	Descoberta canônica prova que nenhum teste ficou órfão.
ADR-011	Mermaid é gerado ou verificado contra catálogo/AST.
ADR-012	Alteração de política e movimentação de código não entram no mesmo passo.
ADR-013	PDF/DOCX usam comparação semântica e visual, não apenas hash bruto.

5. Estados explícitos de fonte

Nenhum caminho pode saltar de “arquivo encontrado” para `final_use_allowed`.

6. Política RED–GREEN–REFACTOR

RED inválido: import quebrado, fixture ausente, sintaxe, ambiente incorreto ou teste já verde sem investigação.

Cada plano TDD implementa um comportamento. Reorganização/configuração/glue usa tarefa executiva, mas sempre com caracterização antes/depois.

7. Planos TDD obrigatórios

TDD-01 — Regimento fail-closed

RED

- regimento fora da pasta bloqueia F3;
- conteúdo curto/incompleto bloqueia;
- fonte, versão, download ou atualização ausentes bloqueiam;
- dois candidatos produzem ambiguous;
- F2 e F3 usam o mesmo catálogo de tribunal.

GREEN

- resolvedor estrito retorna `verified`, `missing`, `ambiguous`, `incomplete` ou `stale`;
- somente `verified` permite avanço.

REFACTOR

- extrair `TribunalCatalog` e `RegimentoVerifier`;
- `forja_sources.py` permanece wrapper.

TDD-02 — Jurisprudência sem autocertificação

RED

- número apenas no nome do PDF não libera uso;
- ocorrência textual genérica não prova identidade;
- tribunal/classe divergente reprova;
- trecho literal ausente reprova;
- fonte indisponível retorna `unverified`, nunca “inexistente”.

GREEN

- pipeline candidato → identidade → conteúdo → uso final;
- `finalUseAllowed` exige identidade e correspondência literal.

REFACTOR

- separar detecção, identificação, literalidade e política.

TDD-03 — Injection scan fail-closed

RED

- exceção produz `P0/unscanned`;
- PDF acima do limite é relatado e bloqueia;
- ilegível não desaparece;
- frase imperativa acadêmica não é executada nem se transforma sozinha em `P0`.

GREEN

- resultado exaustivo por arquivo;
- exit code não zero para material não examinado.

REFACTOR

- `ScanResult` tipado e adaptadores por formato.

TDD-04 — Descoberta completa de testes

RED

- teste sintético órfão reprova a régua;
- coleta demonstra a quebra por `sys.stdout` no import.

GREEN

- remover mutação global de import;
- classificar todos os testes.

REFACTOR

- manifesto único gera suítes rápida, contrato, integração e real.

TDD-05 — Core atômico compatível

Casos:

- JSON ausente, inválido e default permitido são distintos;
- escrita nunca expõe parcial;
- path escape é rejeitado;
- lock preserva timeout/stale;
- timestamps são timezone-aware;
- IDs mantêm formato;
- hashes canônicos permanecem idênticos.

Mover uma primitive por vez e reexportar por `forja_n3_common.py`.

TDD-06 — Catálogos canônicos

Subplanos independentes:

25. TribunalCatalog;
26. PhaseCatalog;
27. ArtifactCatalog;
28. Severity/ReleasePolicy;
29. comandos/feature flags.

Casos: todo artefato tem schema, fase, flag, validador e política; valor duplicado/desconhecido reprova; `generate_n4_contracts.py --check` detecta drift; hashes permanecem quando a mudança é apenas estrutural.

TDD-07 — Revalidação de contexto

Modificar contexto ou inputs após `prepare_attempt` bloqueia promoção antes de efeito canônico. Revalidar `contextHash`, `contractHash` e inputs imediatamente antes do commit.

TDD-08 — Promoção sem parcialidade

Injetar falha entre validação N4, cópia e evento. Nenhuma visão canônica nova fica disponível. Construir `PromotionPlan`, validar antes, promover sob unidade transacional `file-first` e emitir recibo único após sucesso.

TDD-09 — Replay equivalente

- mesmo log → mesmo estado/hash;
- idempotência não duplica;
- concorrência rejeita um escritor;
- evento parcial não é aplicado;
- eventos históricos continuam legíveis.

TDD-10 — Entrega por identidade

- múltiplos DOCX fazem adaptador legado reprovar;
- pacote seleciona um `artifactId` + hash;
- hash divergente bloqueia;
- retry é idempotente;
- draft não conclui demanda.

TDD-11 — Outbox de gestão

- falha do painel não reverte evento do caso;
- outbox persiste;
- retry não duplica;
- evidência de entrega prevalece sobre snapshot atrasado;
- flags desligadas ignoram N4 sem apagar artefatos.

TDD-12 — Render e QA

TDD determinístico para parser Markdown, modelo de blocos e SVG; teste real para Word/PDF.

Casos: todas as páginas inspecionadas; template e metadados preservados; nenhum placeholder; paridade de títulos/parágrafos/tabelas/ressalvas; compositor não aprova o próprio `runId`; regressões Natura, Libra Sul, Patrícia e CORSAN continuam detectadas.

TDD-13 — CLI compatível

Caracterizar argumentos, help, exit code, stdout/stderr sanitizados e efeito de cada wrapper. A nova CLI unifica comportamento; comando antigo delega sem drift.

TDD-14 — Atlas vivo

Alterar catálogo/import sem regenerar ou validar Mermaid deve reprovar. Gerador ignora `var`, cache, outputs, experimentos e projetos estrangeiros.

8. Gates de equivalência

Área	Prova
imports	mesmos símbolos públicos nos módulos antigos
CLI	argumentos, exit codes e efeitos observáveis equivalentes
JSON	schemas e serialização canônica preservados
eventos	replay idêntico para logs congelados
estado	campos/hash equivalentes em movimento puro
contratos	hashes iguais quando não há mudança normativa
artefatos	conteúdo/hash iguais em extrações puras
pacote	mesmo conjunto de IDs e hashes
gestão	sidecar equivalente e nenhuma conclusão falsa
DOCX	blocos, estilos, template, cabeçalho, rodapé e fôlio
PDF	texto, ordem, páginas, geometria e QA
segurança	casos negativos continuam bloqueados
compatibilidade	flags desligadas reproduzem caminho anterior

Correções P0 terão diferenças esperadas explicitamente versionadas; não serão mascaradas como equivalência.

9. Orçamento de arquitetura

- zero ciclo de import;
- `domain` não importa `adapters`, `Word`, `gestão`, `rede` ou `filesystem` concreto;
- função pública nova tem tipos;
- cada CLI é fina;
- complexidade/hotspot não aumenta sem justificativa;
- nenhuma nova fonte de verdade manual paralela;
- toda dependência externa entra por porta;
- todo wrapper registra uso sanitizado.

10. Comandos de verificação previstos

```
$env:PYTHONDONTWRITEBYTECODE = "1"
python -m pytest -q -p no:cacheprovider tests/unit
python -m pytest -q -p no:cacheprovider tests/contract
python -m pytest -q -p no:cacheprovider tests/integration
python -m pytest -q -p no:cacheprovider
python -m ruff check . --no-cache
python -m mypy src/forja
python generate_n4_contracts.py --check
python validate_forja_n3.py --run-replay
python validate_forja_n3.py --real-word --run-replay
python forja_regua.py --rapida
python forja_regua.py
```

Os nomes finais dos flags serão definidos no plano de implementação da régua. Até a nova descoberta existir, cada RED roda isoladamente para não confundir a falha planejada com a coleta global defeituosa.

11. Testes reais obrigatórios

- Word COM e inserção EMF;
- PDF final e render de todas as páginas;

- metadados institucionais;
- pacote e seleção por hash;
- sidecar/outbox de gestão;
- replay de corpus congelado;
- casos prospectivos sem reuso de texto pronto;
- restauração a partir de backup.

12. Rollback

30. trabalhar em baseline isolado;
31. commits RED, GREEN e REFACTOR separados;
32. não apagar eventos, contratos ou artefatos N4;
33. manter fachadas e wrappers;
34. ativar módulos por flag quando necessário;
35. registrar versão/hash em cada execução;
36. desligar flag para retornar ao adaptador anterior;
37. restaurar sidecar a partir de eventos/outbox;
38. não publicar ponteiro canônico em promoção falha;
39. testar restore antes de limpeza física;
40. rollback nunca transforma caso em pronto;
41. contrato novo nunca reinterpreta evento antigo silenciosamente.

13. Definition of Done técnico — G9A

- 100% dos testes descobertos/classificados;
- quatro P0 fail-closed;
- pacote/entrega por identidade e hash;
- eventos e replay preservados;
- promoção atômica;
- catálogos únicos verificáveis;
- fachadas com responsabilidade apenas compatível;
- dependências direcionais e zero ciclo;
- domínio sem infraestrutura;
- CLIs compatíveis ou remoção comprovada;
- outputs/projetos estrangeiros fora da fonte;
- Word/PDF/EMF aprovados em QA real;
- seis replays representativos aprovados e compatibilidade preservada;
- restore demonstrado;
- Mermaid ligado ao código;
- docs, runbooks e manifestos coerentes;
- zero dado ativo ou defesa removida indevidamente.

14. Elegibilidade de promoção — G9B

G9B é deliberadamente separado da conclusão técnica. Exige três ciclos prospectivos completos, mutação semântica $\geq 0,8$ nas famílias aplicáveis, controles benignos, decisão normativa própria e autorização correspondente. Enquanto G9B estiver pendente, nenhum shim, flag ou compatibilidade cuja segurança dependa desses ciclos pode ser removido; N4 permanece `pilot_blocking`.

5. Roadmap — ondas, dependências e gates

Fonte canônica: 03-ROADMAP_REFATORACAO_FORJA.md

ROADMAP — FORJA R1: Refatoração Estrutural Segura

Versão: R1.0-plan

Status: execução não iniciada

Estimativa de engenharia: 28–48 dias úteis, mais a janela dos ciclos reais

Regra: esforço é faixa de planejamento, não promessa de calendário

1. Não replanejar

- N2 M0–M5 já foram concluídos.
- U1–U8 do plano estado da arte já foram implementados; U9/U10 continuam condicionais.
- N3 já possui event store, state machine, attempts, contexto, pacote, QA e gestão; extrair e preservar, não recriar.
- N4 M0–M5 e M6.1–M6.3 já existem em sombra/piloto.
- N4 M6.4/M6.5 ainda dependem de ciclos prospectivos; não promover durante R1.
- Alertas, LocalContext, métricas, ordem de pareceres, mutação semântica, ledger e QA já têm implementações; tratar apenas integração/refatoração comprovada.
- Projeto 3D descartado não será reaberto.
- RAG, LLM-as-judge, DataJud/Sinapses, firewall genérico e protocolo automático não entram.

2. Dependências

Paralelismo permitido:

- R3-A e R3-B, com ownership de arquivos separado;
- R6-A, R6-B e R6-C após estabilização das fachadas.

Não paralelizar:

- promoção/state machine no mesmo arquivo;
- movimentação física antes das fachadas;
- alteração de política junto com extração estrutural.

3. R0 — Baseline, isolamento e contrato

Esforço: 1–2 dias

Dependência: pesquisa concluída

Entrada

- .planning/codebase e este pacote legíveis;
- manifesto/configuração atuais válidos;
- nenhuma autorização implícita de limpeza destrutiva.

Trabalho

- criar branch/worktree isolado;
- registrar itens sujos por categoria;
- inventariar código, testes, contratos, schemas, estado e integrações;
- salvar hashes de fontes protegidas;
- criar backup verificável e ensaio de restore;
- caracterizar CLIs, flags, eventos, schemas e outputs;
- classificar cada item: preservar, extrair, adaptar, arquivar, excluir depois.

Gate G0

- baseline reproduzível;
- backup e restore demonstrados;
- zero trabalho do usuário perdido;
- flags inalteradas;

- manifesto de migração pronto.

Rollback

Abandonar worktree/branch; produção permanece intacta.

4. R1 — Escudo de confiança por TDD

Esforço: 2–4 dias

Bloqueia: todas as ondas estruturais

Trabalho

- descoberta canônica dos testes;
- classificar todos os arquivos em `unit`, `contract`, `integration`, `mutation`, `real`;
- remover mutação de `sys.stdout` no `import`;
- atualizar teste de gestão desatualizado;
- falhar se teste ficar órfão;
- TDD regimento fail-closed;
- TDD fonte jurisprudencial sem autocertificação;
- TDD scanner de injection fail-closed;
- caracterizar `contextHash`, promoção parcial, tribunais e geração `--check`.

Gate G1

- 100% dos testes descobertos/classificados;
- pares `DEVE_PEGAR` / `NÃO_PODE_TRAVAR` verdes;
- quatro P0 corrigidos;
- bateria real separada e identificada;
- régua relata exatamente executados e omitidos.

Rollback

Commits por gate; reverter correção isolada sem tocar em contratos históricos.

5. R2 — Ambiente reproduzível e esqueleto modular

Esforço: 1–2 dias

Trabalho

- `pyproject.toml` e versão Python suportada;
- grupos `core`, `word`, `visual`, `science`, `dev`;
- `pytest`, `Ruff` e `typing` gradual;
- lock reproduzível;
- `src/forja` e `entrypoint` futuro;
- verificador de capacidades externas;
- wrappers antigos permanecem ativos;
- baseline `Ruff` controlado, sem autofix massivo.

Gate G2

- ambiente limpo instala e executa suíte não-real;
- testes/imports independem do diretório atual;
- nenhum comando antigo quebra;
- capacidade ausente é declarada, não mascarada.

Rollback

Remover configuração/esqueleto; scripts antigos continuam autossuficientes.

6. R3 — Core e fontes de verdade

Esforço: 4–7 dias

R3-A — Core

Extrair JSON atômico, collections, tempo, IDs, hashes, locks, paths e erros. `forja_n3_common.py` permanece fachada.

R3-B — Domínio

Centralizar fases, tribunais/CNJ, severidades, comandos, flags, estados epistemológicos e `ArtifactCatalog`.
Gerar/verificar schemas, contratos derivados, índices, métricas, documentação e invalidação.

Gate G3

- clones utilitários substituídos por fachada;
- comportamento equivalente;
- catálogo cobre 100% dos artefatos;
- `--check` sem drift;
- contratos vigentes/candidatos continuam separados;
- nenhuma defesa independente removida.

Rollback

Fachadas apontam à implementação antiga; catálogos novos podem ser desligados.

7. R4 — Resolução, estado e promoção transacional

Esforço: 3–5 dias

Trabalho

- resolvedor estrito N3 canônico;
- glob legado apenas adaptador com ambiguidade explícita;
- escritas diretas substituídas por portas atômicas;
- revalidar `contractHash`, `contextHash` e inputs;
- pipeline `prepare` → `validate` → `commit`;
- evento/visão canônica apenas após gates;
- falha preserva `attempt`, não publica parcialidade;
- eventos históricos/replay compatíveis;
- `resolvedAt/resolvedBy` em eventos.

Gate G4

- replay determinístico;
- concorrência rejeita writer antigo;
- cada fronteira suporta interrupção/retomada;
- falha N4 não altera visão canônica;
- zero regressão silenciosa.

Rollback

Porta transacional é desligada; eventos novos permanecem preservados/legíveis.

8. R5 — Entrega e gestão

Esforço: 3–5 dias

Trabalho

- `artifactId` + `sha256` vira identidade de entrega;
- heurística N2 fica somente em descoberta legada;
- unificar `package/close/delivery` integrity por serviço;
- `ManagementOutbox` durável e idempotente;
- gestão sai da transação principal;
- evidência de entrega prevalece sobre snapshot atrasado;
- distinguir escritório de protocolo judicial;
- retries não duplicam draft, comentário ou evidência.

Gate G5

- arquivo entregue é exatamente o auditado;
- pane de gestão não corrompe caso;
- outbox sincroniza uma vez;
- draft não conclui demanda;

- rollback N2/N3 demonstrado.

9. R6 — Decomposição dos hotspots

Esforço: 5–8 dias

R6-A — Render/visual

Separar parsing, layout, emissão, Word COM e QA em `forja_visual.compor` e `forja_render.docx.render`. Criar `DocumentRenderer` e preservar adaptador Medina.

R6-B — Validação

Decompor `forja_n4.validate.validate_case`, PSO, auditoria adversarial e regras contextuais em validadores compostos.

R6-C — CLI/orquestração

Separar em dois planos: P13A estabiliza framework, `doctor`, `qa` e exit codes sem tocar wrappers de serviço; P13B, somente após P09–P12, migra wrappers por allowlist, um a um, sem arquivo compartilhado com outros owners.

Gate G6

- APIs públicas tipadas;
- biblioteca não chama `sys.exit`;
- Word/PDF/EMF reais equivalentes;
- QA de todas as páginas;
- mutação, replay e telemetria reais verdes;
- corpus representativo sem regressão material.

Rollback

Wrapper por hotspot retorna à implementação antiga. A cadeia visual nunca é substituída de uma vez.

10. R7 — Sanitização física

Esforço: 3–5 dias + backup

Trabalho

- testes → `tests/`;
- geradores → `tools/`;
- pilotos → `experiments/n4/`;
- scripts datados → `migrations/`;
- outputs → `var/` por configuração;
- retirar cópia aninhada do FocoEdital apenas após backup/comparação/ausência de referência/link/restore;
- política de retenção de cache, telemetria, reports e renders;
- preservar estados, eventos, contratos e invalidados.

Gate G7

- árvore fonte pequena/navegável;
- imports/CLIs antigos funcionam por wrappers;
- zero referência quebrada;
- backup/restore comprovados;
- telemetria identifica acessos legados.

Rollback

Manifesto de movimento restaura origem/destino por hash.

11. R8 — CLI única e atlas vivo

Esforço: 3–5 dias

Trabalho

- endpoint `forja` com subcomandos;
- wrappers emitem telemetria;
- separar arquitetura, ADRs, operações e changelog;

- gerar/verificar diagramas contra AST/catálogos;
- matriz alterar $X \rightarrow$ verificar Y ;
- PDF técnico atualizado;
- links/Mermaid validados automaticamente.

Gate G8

- IA encontra módulo, fonte, testes e impacto em poucos passos;
- 100% Mermaid renderiza;
- grafos correspondem ao código;
- PDF legível/inspecionado;
- documentação não afirma promoção inexistente.

12. R9 — Replay, canários e compatibilidade

Esforço de engenharia: 3–5 dias

Calendário: depende de demandas reais

Trabalho

- seis replays representativos;
- três ciclos prospectivos reais;
- N4 permanece `pilot_blocking` durante a avaliação;
- mutação semântica por família, sem mascarar fraqueza;
- medir falsos bloqueios benignos;
- pareceres Helena/Cícero antes da redação;
- regimento e ledger material comprovados;
- flags N3 uma a uma;
- shim removido apenas após janela sem uso.

Gate G9A — conclusão técnica da refatoração

- zero divergência evento/estado/pacote/gestão/evidência;
- zero P0 conhecido escapando;
- controles benignos publicados;
- rollback demonstrado;
- seis replays representativos aprovados;
- shims necessários e compatibilidade dependente de ciclo prospectivo preservados;
- decisão normativa separada para qualquer promoção.

G9A encerra tecnicamente R1. Arquitetura limpa não promove N4.

Gate G9B — elegibilidade de cutover/promoção

- três ciclos prospectivos completos;
- mutação semântica $\geq 0,8$ nas famílias aplicáveis;
- controles benignos e falsos bloqueios avaliados;
- decisão normativa própria e autorização correspondente.

G9B pode permanecer pendente sem invalidar G9A. Enquanto pendente, proíbe remover compatibilidade ou alterar flags que dependam da prova prospectiva; N4 continua `pilot_blocking`.

13. Visão temporal

A data inicial é meramente ilustrativa para visualizar dependências. Execução só começa mediante comando posterior.

14. Riscos e controles

Risco	Controle
workspace sujo	worktree, hashes e backup
remover defesa independente	classificar duplicação antes de extrair
quebrar histórico	replay e schemas versionados
arquivo errado	ID + hash; nunca primeiro glob

Risco	Controle
promoção informal	flags e decisão normativa separadas
testes sintéticos verdes	corpus real Word/PDF/telemetria
automação oculta quebrada	wrappers + telemetria + janela sem uso
documentação obsoleta	geração/verificação automática
autofix semântico	Ruff incremental após caracterização
limpeza destrutiva	decisão item a item, backup e restore
gestão dentro da transação	outbox idempotente
citação/regimento degradados	P0 fail-closed antes da arquitetura

15. Critério de conclusão do programa

Todos os gates G0–G9A devem ter evidência para classificar a refatoração estrutural como concluída. G9B governa apenas elegibilidade de cutover/promoção. Sua pendência não reabre G9A, mas bloqueia qualquer remoção de compatibilidade ou alteração de flag dependente de prova prospectiva.

6. Rastreabilidade ponta a ponta

Fonte canônica: 05-MATRIZ_RASTREABILIDADE.md

Matriz de rastreabilidade — FORJA R1

Objetivo: provar que cada requisito possui plano, teste, gate e evidência esperada

1. Requisitos funcionais

Requisito	Onda/plano	Teste ou verificação	Evidência de conclusão
RF-REF-001 baseline	R0 / P00	inventário + restore	manifest, hashes, log de restore
RF-REF-002 régua	R1 / P01	teste órfão reprova	relatório 100% classificado
RF-REF-003 P0	R1 / P02–P04	negativos/benignos	quatro gates fail-closed
RF-REF-004 reproduzibilidade	R2 / P05	instalação limpa	lock + suíte não-real
RF-REF-005 fachadas	R2–R7	contratos de import/CLI	equivalência e telemetria
RF-REF-006 catálogos	R3-B / P07	cobertura de catálogo	zero definição órfã
RF-REF-007 geração	R3-B / P07	--check	geração determinística
RF-REF-008 event store	R4 / P08	replay/concorrência	stateHash/equivalência
RF-REF-009 promoção	R4 / P08	fault injection	zero parcialidade
RF-REF-010 resolução	R4 / P08	0/1/N matches	ambiguidade explícita
RF-REF-011 entrega	R5 / P09	dois DOCX/hash divergente	receipt por ID/hash
RF-REF-012 outbox	R5 / P10	pane/retry	sync uma vez
RF-REF-013 render	R6-A / P11	Word/PDF/EMF	QA todas as páginas
RF-REF-014 hotspots	R6 / P11–P13	caracterização	complexidade reduzida
RF-REF-015 separação	R7 / P14	imports/paths	fonte sem outputs
RF-REF-016 sanitização	R7 / P14	manifesto/restore	zero perda
RF-REF-017 CLI	R6-C / P13A–P13B	args/help/exit/effect + allowlist	framework e wrappers equivalentes
RF-REF-018 erros	R2/R6-C	teste de mapeamento	biblioteca sem SystemExit
RF-REF-019 Mermaid	R8 / P15	render 100%	atlas validado
RF-REF-020 navegação IA	R8 / P15	roteiro de localização	matriz de impacto completa
RF-REF-021 remoção shims	R9 / P16	telemetria/referências	evidência individual
RF-REF-022 F2-A	R1/R9	corpus F2-A	contrato/legacy preservados

2. RNFs

RNF	Gates	Métrica
RNF-001 segurança jurídica	G1, G6, G9A/G9B	zero gate degradado
RNF-002 compatibilidade	G3–G9A; G9B para promoção	100% corpus legível
RNF-003 auditabilidade	G0–G9	todo movimento com hash/razão
RNF-004 fail-closed	G1	erro nunca vira ok
RNF-005 determinismo	G3/G4/G8	execuções equivalentes
RNF-006 modularidade	G2/G3/G6	zero import proibido
RNF-007 manutenibilidade	G6	tipos/complexidade não pioram
RNF-008 reproduzibilidade	G2	ambiente limpo verde
RNF-009 desempenho	G4–G9	regressão dentro do orçamento
RNF-010 portabilidade	G2/G6	core sem Word
RNF-011 observabilidade	G4/G5/G8	IDs/hashes/duração/erro

RNF	Gates	Métrica
RNF-012 privacidade	todos	secret/provenance scan limpo
RNF-013 legibilidade	G8	Mermaid 100%; ≥8 pt
RNF-014 recuperabilidade	todos	restore por onda
RNF-015 limite normativo	todos	manifest preservado

3. Invariantes e sentinelas

Grupo	Invariantes	Sentinela
estado normativo	INV-01, INV-20, INV-21	diff do manifest/config/contratos
fonte e prova	INV-02–INV-08, INV-22–INV-24	gates negativos + fonte oficial
fluxo F2-A/conselho	INV-09–INV-11	corpus F2-A + pareceres
documento final	INV-12–INV-15	Word COM, EMF, PDF, metadata, QA
entrega	INV-16–INV-19	receipt/evidência/outbox

4. Requisito do usuário → resultado

Pedido	Resultado planejado
limpeza/sanitização	R0 + R7, com backup e restore
tirar duplicidade	R3/R5/R6, distinguindo defesa independente
boas práticas	pacote, typing, portas, erros, testes arquiteturais
modularidade	core/domain/application/ports/adapters
alterar sem quebrar	matriz de impacto, TDD, wrappers e replay
fácil para IA	README, atlas vivo, ownership e fontes canônicas
PDF	deliverable consolidado e QA página a página
TDD	14 comportamentos RED–GREEN–REFACTOR
roadmap	ondas R0–R9, gates G0–G9
diagramas	22 diagramas-base + Gantt

5. Cobertura dos planos executáveis

Plano	Tipo	Requisitos principais
P00 baseline	execute	001, 016
P01 régua	tdd	002, 022
P02 regimento	tdd	003, 006
P03 citações	tdd	003, 018
P04 injection	tdd	003, 018
P05 ambiente	execute	004, 005, 018
P06 core	tdd	005, 008
P07 catálogos	tdd	006, 007, 022
P08 estado/promoção	tdd	008, 009, 010
P09 entrega	tdd	011, 013
P10 gestão	tdd	012, 018
P11 render	tdd	013, 014
P12 validadores	tdd	014, 018
P13A CLI framework	execute	005, 017, 018
P13B CLI wrappers	execute	005, 017, 018, 021

Plano	Tipo	Requisitos principais
P14 sanitização física	execute	015, 016
P15 atlas	execute	019, 020
P16 replay/cutover	execute	021, 022

6. Gate de cobertura

O pacote só está pronto para execução se:

- todo RF aparece em pelo menos um plano;
- todo RNF possui gate e métrica;
- todo invariante possui sentinela;
- todo plano possui leitura prévia, ação concreta, verificação e aceite;
- nenhum requisito histórico já implementado foi reespecificado como funcionalidade nova.

7. Testes, rollback e cutover

Fonte canônica: 06-TESTES_ROLLBACK_E_CUTOVER.md

Estratégia de testes, rollback e cutover — FORJA R1

1. Princípio

Nenhuma limpeza é concluída por ausência de erro aparente. Cada onda deve provar preservação, diferenças intencionais, recuperabilidade e cobertura do próprio verificador.

2. Camadas de teste

Camada	Escopo	Frequência
unidade	funções puras, reducers, parsers, catálogos	toda tarefa
contrato	schemas, geração, wrappers, CLI, F2-A	toda onda
integração	filesystem, locks, events, promotion, outbox	toda onda afetada
mutação/adversarial	citações, contexto, parte, valor, pedido	gates jurídicos
real	Word COM, EMF, PDF, páginas, telemetria	R1 baseline, R6 e R9

3. Descoberta da régua

O manifesto da suíte é parte do produto testado. Um teste novo sem classificação deve tornar a validação vermelha.

4. Testes P0

Regimento

- correto e completo passa;
- ausente, fora do caso, ambíguo, curto, sem fonte, sem versão, sem download ou sem emendas bloqueia;
- tribunal divergente bloqueia;
- `_LEIS_GERAIS` ausente bloqueia a liberação aplicável.

Citação

- nome de arquivo não prova identidade;
- número em texto genérico não prova fonte;
- tribunal/classe divergentes bloqueiam;
- trecho inexistente bloqueia;
- indisponibilidade vira `unverified`;
- fonte revogada invalida uso.

Injection

- exceção, arquivo ilegível, grande ou não suportado vira `unscanned P0`;
- todo input aparece no resultado;
- conteúdo imperativo externo nunca vira comando;
- citação acadêmica benigna não trava sem sinal técnico.

Régua

- teste órfão trava;
- `stdout` global não é trocado no import;
- N2/N3/N4/F2-A aparecem no relatório;
- bateria real nunca é reportada como executada quando omitida.

5. Oráculos de equivalência

Objeto	Comparação
evento	sequência, tipo, payload canônico e revisão
estado	campos e hash normalizado

Objeto	Comparação
contrato/schema	hash bruto quando estruturalmente inalterado
JSON gerado	conteúdo canônico sem timestamp volátil
CLI	argumentos, exit, stdout/stderr sanitizados, efeitos
pacote	IDs, hashes, bytes e anexos
DOCX	estrutura OOXML, estilos, blocos, tabelas, metadados
PDF	texto, páginas, geometria, imagens e QA visual
gestão	sidecar, precedência, links e idempotência

Correções de P0 têm diferenças esperadas registradas; somente caminhos inseguros devem mudar.

6. Fault injection

Injetar falhas em:

42. leitura de input;
43. lock;
44. escrita de artifact staging;
45. revalidação de contexto;
46. validação N4;
47. cópia/promote;
48. append do evento;
49. materialização do snapshot;
50. enqueue da gestão;
51. flush da outbox;
52. Word COM;
53. exportação PDF;
54. render de página.

Para cada ponto, declarar estado permitido, artefatos preservados, retry e rollback.

7. Corpus

Replays mínimos

- Azimut — isolamento e fontes pendentes;
- CORSAN — cartões, overflow e timeline;
- Libra Sul — memoriais longos e sobreposição;
- Natura — sequência visual e colisões;
- Patrícia/Fábio — números, caixas e JSON;
- Plano de Saúde — pacote múltiplo e regressão de fase.

Cada replay usa cópia imutável e não reescreve peça histórica.

Ciclos prospectivos

Três casos novos, com F2-A, regimento, conselho, redação, auditoria, QA e entrega. Texto final preexistente não pode ser usado como oráculo de geração.

8. Orçamento de desempenho

Baseline deve medir:

- descoberta e suíte rápida;
- suíte não-real completa;
- replay por caso;
- validação N4;
- render DOCX/PDF;
- sincronização de gestão.

Regressão >15% exige explicação e aceite técnico; regressão >30% bloqueia a onda, salvo correção de segurança que justifique explicitamente o custo.

9. Backup e restore

Antes de mover/apagar:

- inventário com tamanho/função/consumidor;
- hash de cada arquivo protegido;
- backup fora do destino da limpeza;
- manifest origem → destino;
- comando de restauração;
- ensaio em diretório temporário;
- comparação pós-restore;
- decisão manter, mover, arquivar, excluir depois.

Nunca apagar diretamente durante R0–R6.

10. Rollback por onda

Onda	Rollback
R0	abandonar baseline isolado
R1	reverter gate isolado; manter testes RED como evidência
R2	remover manifesto/esqueleto; usar scripts atuais
R3	fachada aponta para implementação antiga
R4	desligar porta transacional; preservar eventos
R5	usar rota anterior; reprocessar outbox
R6	wrapper retorna hotspot antigo
R7	restaurar por manifest/hash
R8	docs/CLI antiga continuam disponíveis
R9	manter shims e flags no estado anterior

11. Cutover

Refatoração estrutural e promoção N3/N4 são decisões diferentes. Um módulo pode se tornar padrão interno sem mudar a especificação jurídica, desde que o manifest e as flags expressem isso corretamente.

12. Gate final técnico — G9A

- suíte completa e cobertura da régua;
- replay equivalente;
- Word/PDF/EMF e todas as páginas;
- outbox/entrega por hash;
- segurança e segredos;
- links e Mermaid;
- backup/restore;
- manifestos e runbooks coerentes;
- nenhum shim removido sem prova;
- nenhuma promoção normativa implícita.

13. Gate de elegibilidade — G9B

Somente se houver intenção posterior de cutover/promoção:

- três ciclos prospectivos reais completos;
- mutação semântica $\geq 0,8$ nas famílias aplicáveis;
- falsos bloqueios benignos medidos;
- decisão normativa própria e autorização correspondente.

G9B pendente não impede fechar tecnicamente a refatoração em G9A, mas mantém shims, flags e compatibilidade cuja segurança dependa dos ciclos prospectivos.

8. Catálogo dos 18 planos executáveis

FORMATO DE EXECUÇÃO

Cada plano possui contexto prévio, ameaça, arquivos a ler antes de editar, ação concreta, critério de aceite, comando de verificação e condição de rollback.

P00-BASELINE-PLAN

Requisitos	RF-REF-001, RF-REF-016, RNF-003, RNF-014
------------	--

Objetivo: Criar um baseline isolado, reproduzível e restaurável antes de qualquer alteração funcional.

Ameaça controlada: Riscos: sobrescrever trabalho existente, copiar segredo para relatório, backup incompleto e restore não testado. Controles: worktree separado, inventário sanitizado, hashes, backup privado e restauração em diretório temporário.

Verificação: - validar JSON dos manifests; - executar scanner de segredos nos relatórios; - comparar status/timestamps do workspace original; - anexar log do restore com contagens e hashes.

P01-TEST-DISCOVERY-PLAN

Requisitos	RF-REF-002, RF-REF-022, RNF-008
------------	---------------------------------

Objetivo: Garantir que a régua descubra e classifique todos os testes e nunca produza falso verde por omissão.

Ameaça controlada: Risco: marcar teste como executado sem executá-lo, ocultar teste real caro ou coletar código com efeito colateral. O manifesto é validado contra o filesystem e o runner grava comandos/resultados reais.

Verificação: Executar teste de descoberta isolado, coleta global, suíte rápida e validação N3; verificar que a bateria real não é alegada quando omitida.

P02-REGIMENTO-PLAN

Requisitos	RF-REF-003, RF-REF-006, RNF-001, RNF-004
------------	--

Objetivo: Tornar o gate de regimento inequivocamente fail-closed e unificar o catálogo de tribunais.

Ameaça controlada: Risco jurídico crítico: selecionar primeiro arquivo de outro caso ou degradar falta material para P1. Qualquer ambiguidade, erro ou ausência é bloqueante.

Verificação: Executar negativos/benignos, suíte F2/F3 e caso-piloto sobre cópia; revisar relatório sem mencionar proveniência operacional na peça.

P03-CITATION-PLAN

Requisitos	RF-REF-003, RF-REF-018, RNF-001, RNF-004
------------	--

Objetivo: Impedir que presença nominal de número em arquivo autocertifique jurisprudência para uso final.

Ameaça controlada: Risco: citar decisão errada ou atribuir frase a autoridade incorreta. Fonte indisponível nunca vira inexistência nem aprovação.

Verificação: Executar regressões de veneno, controles benignos, F7 e auditoria adversarial; confirmar zero fonte autocertificada.

P04-INJECTION-PLAN

Requisitos	RF-REF-003, RF-REF-018, RNF-004, RNF-012
------------	--

Objetivo: Fazer o scanner registrar e bloquear todo material que não conseguiu examinar, sem tratar conteúdo externo como comando.

Ameaça controlada: Riscos: entrada maliciosa não analisada ser aceita; citação acadêmica benigna gerar falso bloqueio. Separar sinal técnico de linguagem imperativa contextual.

Verificação: Executar poison/benign suite, arquivo grande sintético e scanner de segredos no relatório.

P05-ENVIRONMENT-PLAN

Requisitos	RF-REF-004, RF-REF-005, RF-REF-018, RNF-006, RNF-008, RNF-010
------------	---

Objetivo: Criar ambiente reproduzível e esqueleto modular sem mover ainda o runtime atual.

Ameaça controlada: Risco: lock incompatível com Word/Windows ou instalação global mascarar dependência ausente. Validar em ambiente limpo e separar grupos opcionais.

Verificação: Instalar a partir do lock em ambiente limpo; executar suíte não-real, doctor e contratos de CLI; comparar baseline Ruff sem autofix.

P06-CORE-FACADE-PLAN

Requisitos	RF-REF-005, RF-REF-008, RF-REF-018, RNF-005, RNF-006
------------	--

Objetivo: Extrair primitivas neutras preservando integralmente a API observável de `forja_n3_common.py`.

Ameaça controlada: Riscos: mudar hash, atomicidade, timeout/stale, timezone ou path confinement. Golden tests congelam cada invariante antes da extração.

Verificação: Suíte core, state machine, run/package, replay de fixture e análise de imports.

P07-CATALOGS-PLAN

Requisitos	RF-REF-006, RF-REF-007, RF-REF-022, RNF-005, RNF-006
------------	--

Objetivo: Criar fontes canônicas tipadas e eliminar drift de tribunais, fases, artefatos, severidade e geração.

Ameaça controlada: Risco: catálogo novo omitir exceção contextual ou colapsar contratos vigente/candidato. Teste de cobertura e separação por specVersion são bloqueantes.

Verificação: Testes de catálogo, schemas, contratos, F2-A, N4 agregada e duas execuções de geração comparadas.

P08-STATE-PROMOTION-PLAN

Requisitos	RF-REF-008, RF-REF-009, RF-REF-010, RNF-002, RNF-004, RNF-005
------------	---

Objetivo: Unificar resolução e tornar promoção de attempts transacional, revalidada e recuperável.

Ameaça controlada: Risco crítico: estado parcial, case errado ou reinterpretção histórica. Fault injection e expectedRevision bloqueiam efeitos ambíguos.

Verificação: Fault injection completo, concorrência, replay histórico, retomada após interrupção e suíte package/N4.

P09-DELIVERY-IDENTITY-PLAN

Requisitos	RF-REF-011, RF-REF-013, RNF-002, RNF-004
------------	--

Objetivo: Selecionar e fechar entrega exclusivamente por identidade auditada e hash.

Ameaça controlada: Risco jurídico/operacional: enviar versão errada ou considerar draft como entrega. A identidade do pacote é imutável e vinculada à evidência.

Verificação: Suíte package/delivery, hash tamper, idempotência e caso real controlado sem envio externo.

P10-MANAGEMENT-OUTBOX-PLAN

Requisitos	RF-REF-012, RF-REF-018, RNF-004, RNF-011
------------	--

Objetivo: Desacoplar gestão da transação do caso por outbox idempotente e preservar precedência de evidência.

Ameaça controlada: Riscos: perda/duplicação de status, falsa conclusão e exposição de conversa. Payload mínimo e sanitizado; demanda não conclui por draft.

Verificação: Falhas/retries, sidecar real em diretório temporário, testes de gestão e scanner de privacidade.

P11-RENDER-PIPELINE-PLAN

Requisitos	RF-REF-013, RF-REF-014, RNF-006, RNF-010, RNF-013
------------	---

Objetivo: Decompor renderização em parser, modelo, composição, Word e QA sem regressão visual ou factual.

Ameaça controlada: Riscos: omitir texto/tabela, substituir EMF por raster, metadata errada ou autoaprovação. Comparação estrutural e inspeção de todas as páginas são bloqueantes.

Verificação: Unidade/parser, integração fake, Word COM real, EMF, PDF, metadata, render/inspeção de todas as páginas e fidelity MD/DOCX/PDF.

P12-VALIDATION-PIPELINE-PLAN

Requisitos	RF-REF-014, RF-REF-018, RNF-001, RNF-004, RNF-006
------------	---

Objetivo: Transformar agregadores extensos em pipeline explícito de validadores puros/contextuais sem relaxar severidades.

Ameaça controlada: Risco: decomposição remover recálculo independente ou mudar P0/P1. Golden reports e mutation tests sentinelas preservam comportamento; fail-open é proibido.

Verificação: Golden reports, mutation/adversarial, N4 completa, F2-A, PSO e anti-autocertificação.

P13A-CLI-FRAMEWORK-PLAN

Requisitos	RF-REF-005, RF-REF-017, RF-REF-018, RNF-007
------------	---

Objetivo: Estabilizar o framework da CLI, contratos, doctor, qa e exit codes sem migrar wrappers dependentes de serviços ainda instáveis.

Ameaça controlada: Riscos: framework incorporar regra jurídica, biblioteca chamar SystemExit e contrato mascarar capacidade ausente. P13A toca somente a allowlist declarada e não expõe `validate|package|close|deliver`.

Verificação: Contratos do framework, allowlist de diff, secret scan de logs, suíte e execução em cwd diferentes.

P13B-CLI-WRAPPERS-PLAN

Requisitos	RF-REF-005, RF-REF-017, RF-REF-018, RF-REF-021, RNF-007, RNF-011
------------	--

Objetivo: Migrar wrappers antigos, um por vez, somente depois de os serviços de estado, entrega, gestão, render e validação estarem estabilizados.

Ameaça controlada: Riscos: automação oculta depender de argumento/exit/stdout, arquivo compartilhado com outro owner e log expor dados. A allowlist nasce de `CLI_CONTRACTS.json`; um wrapper por tarefa/commit; qualquer diff externo bloqueia.

Verificação: Contratos de todos os CLIs, diff contra allowlist, secret scan de logs, suíte, replay de efeitos e execução em cwd diferentes.

P14-PHYSICAL-SANITIZATION-PLAN

Requisitos	RF-REF-015, RF-REF-016, RF-REF-021, RNF-003, RNF-014
------------	--

Objetivo: Separar código, testes, ferramentas, experimentos, histórico e outputs sem perda ou quebra de automação.

Ameaça controlada: Risco crítico de apagar estado, segredo, evidência, skill ativa ou projeto útil. Movimentos seguem manifesto/hash; exclusões exigem checkpoint humano e restore testado.

Verificação: Manifest/hash antes/depois, descoberta canônica, suíte completa, CLIs, links, subprocessos, smoke de runtime, secret scan e restore de amostra por classe e por raiz ativa.

P15-LIVE-ATLAS-PLAN

Requisitos	RF-REF-019, RF-REF-020, RNF-005, RNF-013
------------	--

Objetivo: Criar atlas vivo, matriz de impacto e navegação segura para humanos e IAs.

Ameaça controlada: Risco: documentação bonita ficar desatualizada ou incluir segredos/outputs. Gerador usa AST/catálogos e allowlist de metadados, ignorando var/archive/foreign.

Verificação: Teste de drift, links, secret scan, mmdc, PDF/PNG page QA e onboarding por agente novo em modo leitura.

P16-REPLAY-CUTOVER-PLAN

Requisitos	RF-REF-021, RF-REF-022, RNF-001, RNF-002, RNF-014, RNF-015
------------	--

Objetivo: Provar equivalência real, ativar caminhos internos gradualmente e remover compatibilidade somente com evidência.

Ameaça controlada: Riscos: falsa promoção N4, regressão em peça real, shim ainda usado, cutover sem rollback. Flags uma a uma, N4 permanece pilot_blocking e toda remoção exige checkpoint.

Verificação: G9A exige seis replays, Word/PDF/telemetria, outbox, restore, secret scan e auditoria independente. G9B acrescenta três ciclos prospectivos e decisão normativa própria.

9. Atlas visual selecionado

COBERTURA VISUAL

O atlas fonte contém 30 blocos Mermaid validados. Esta edição incorpora os 17 diagramas de maior utilidade operacional; os demais permanecem no arquivo canônico e no relatório de renderização.

5. Estados explícitos de fonte

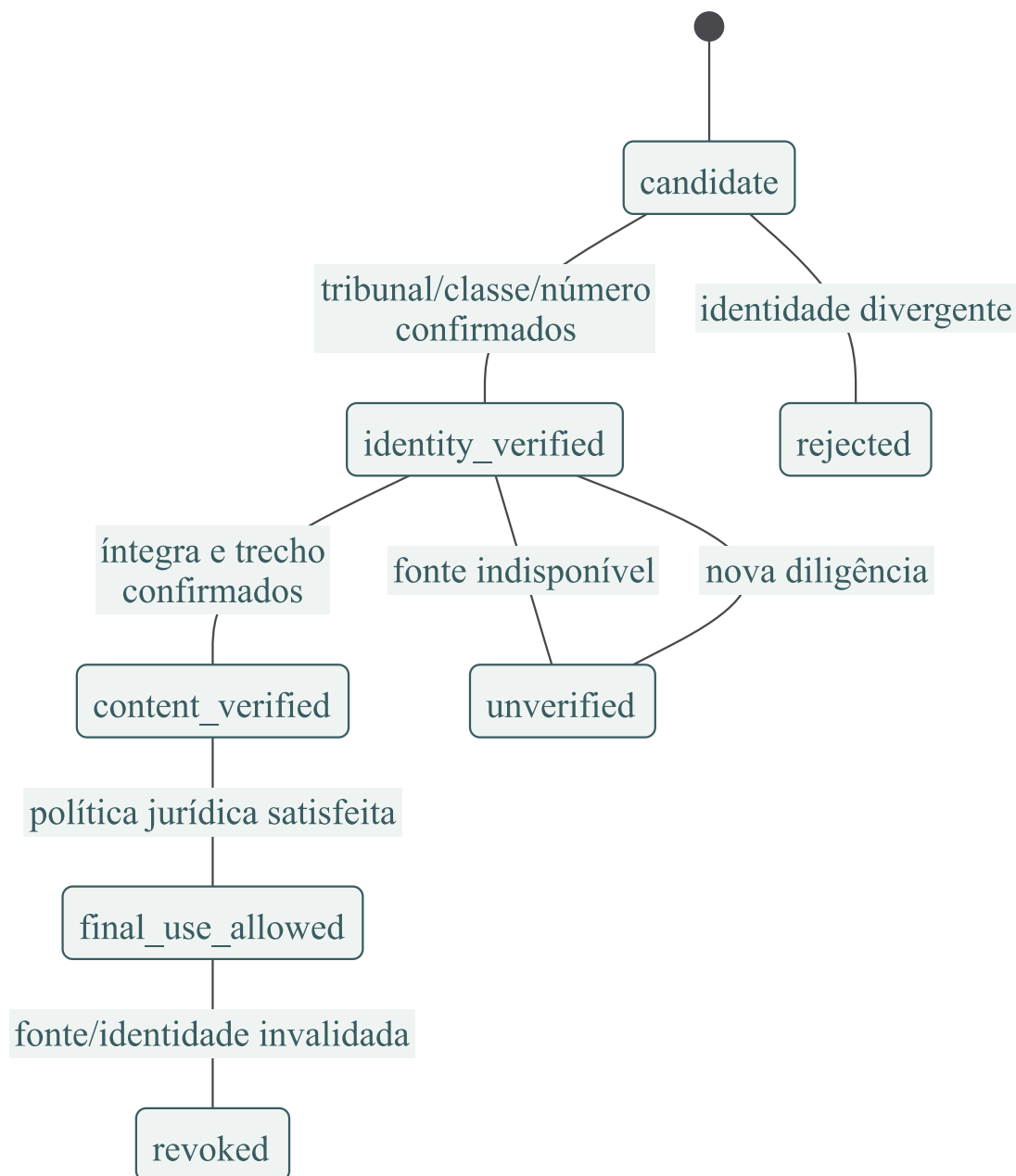


Diagrama S01 — 5. Estados explícitos de fonte • largura final 14,9 cm

6. Política RED–GREEN–REFACTOR

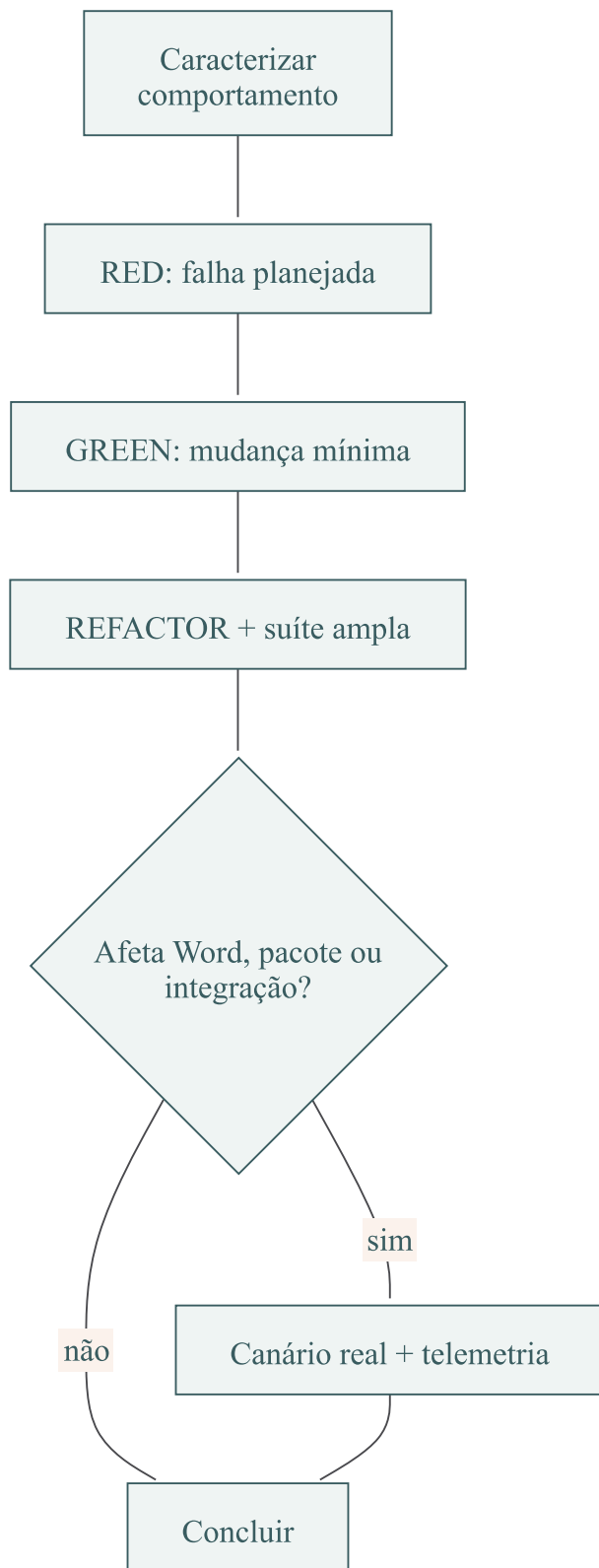


Diagrama S02 — 6. Política RED–GREEN–REFACTOR • largura final 8.4 cm

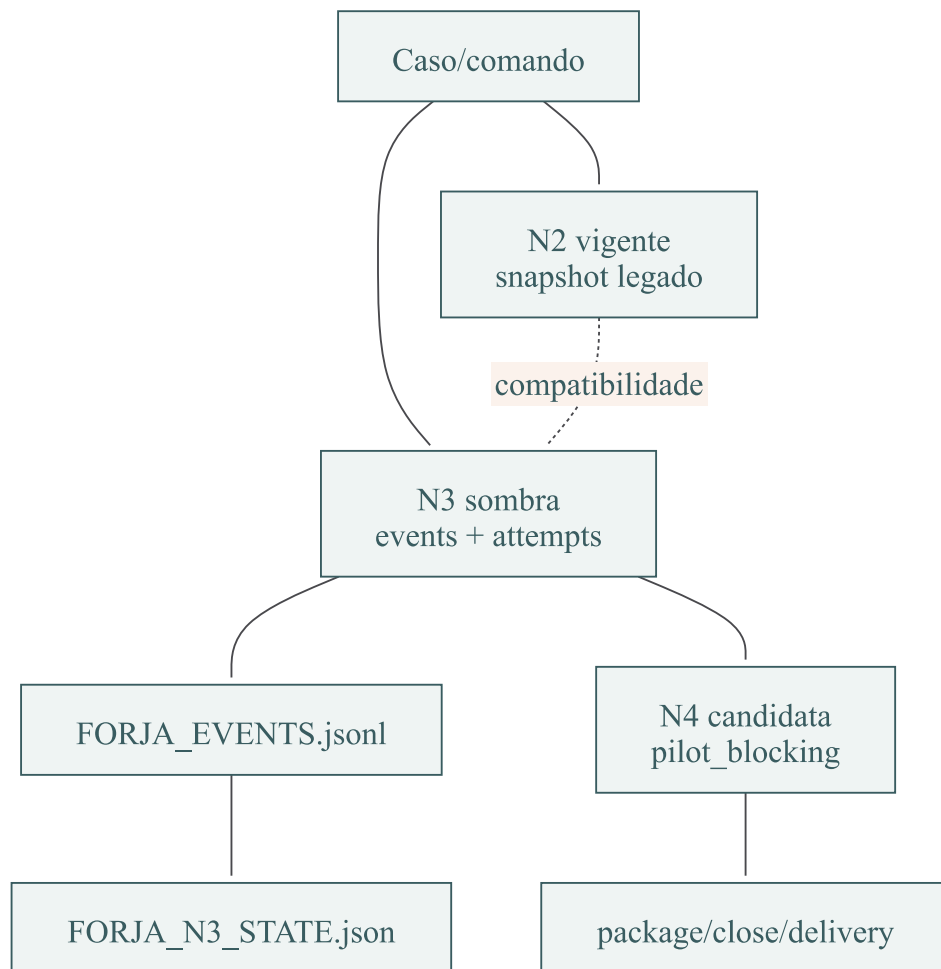
D01 — Estado normativo atual

Diagrama D01 — D01 — Estado normativo atual • largura final 12.9 cm

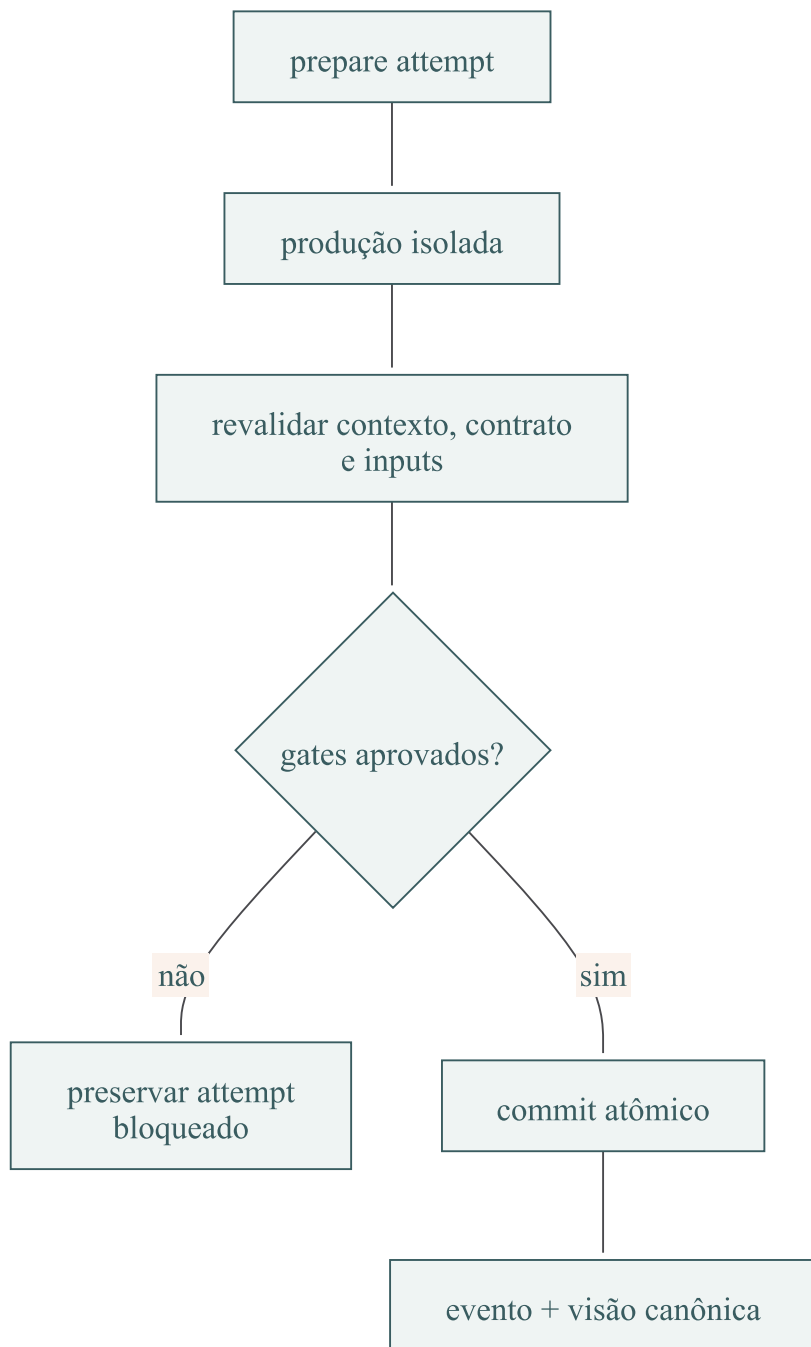
D06 — Attempt e promoção transacional

Diagrama D06 — D06 — Attempt e promoção transacional • largura final 11.1 cm

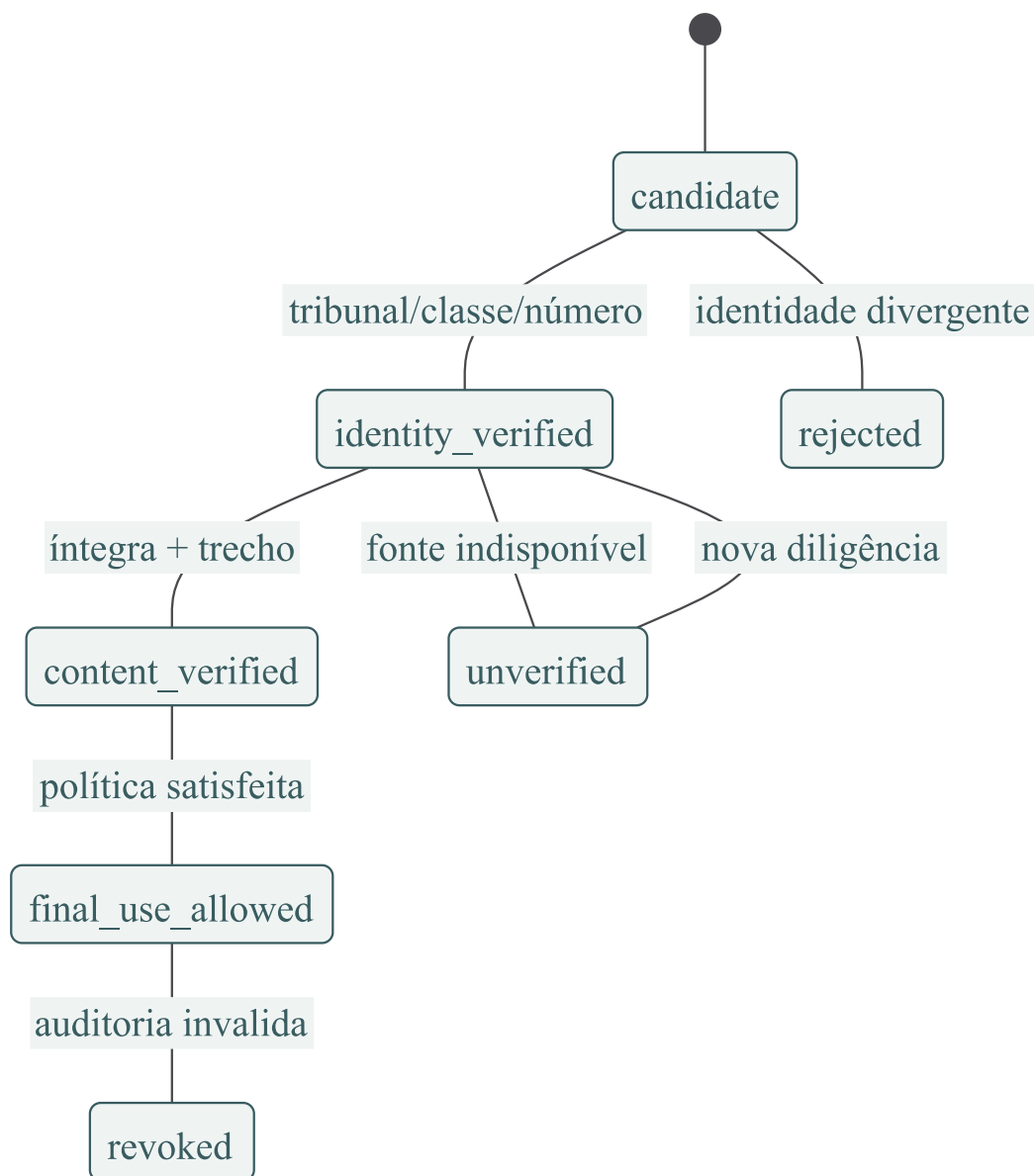
D09 — Verificação de citação

Diagrama D09 — D09 — Verificação de citação • largura final 14.2 cm

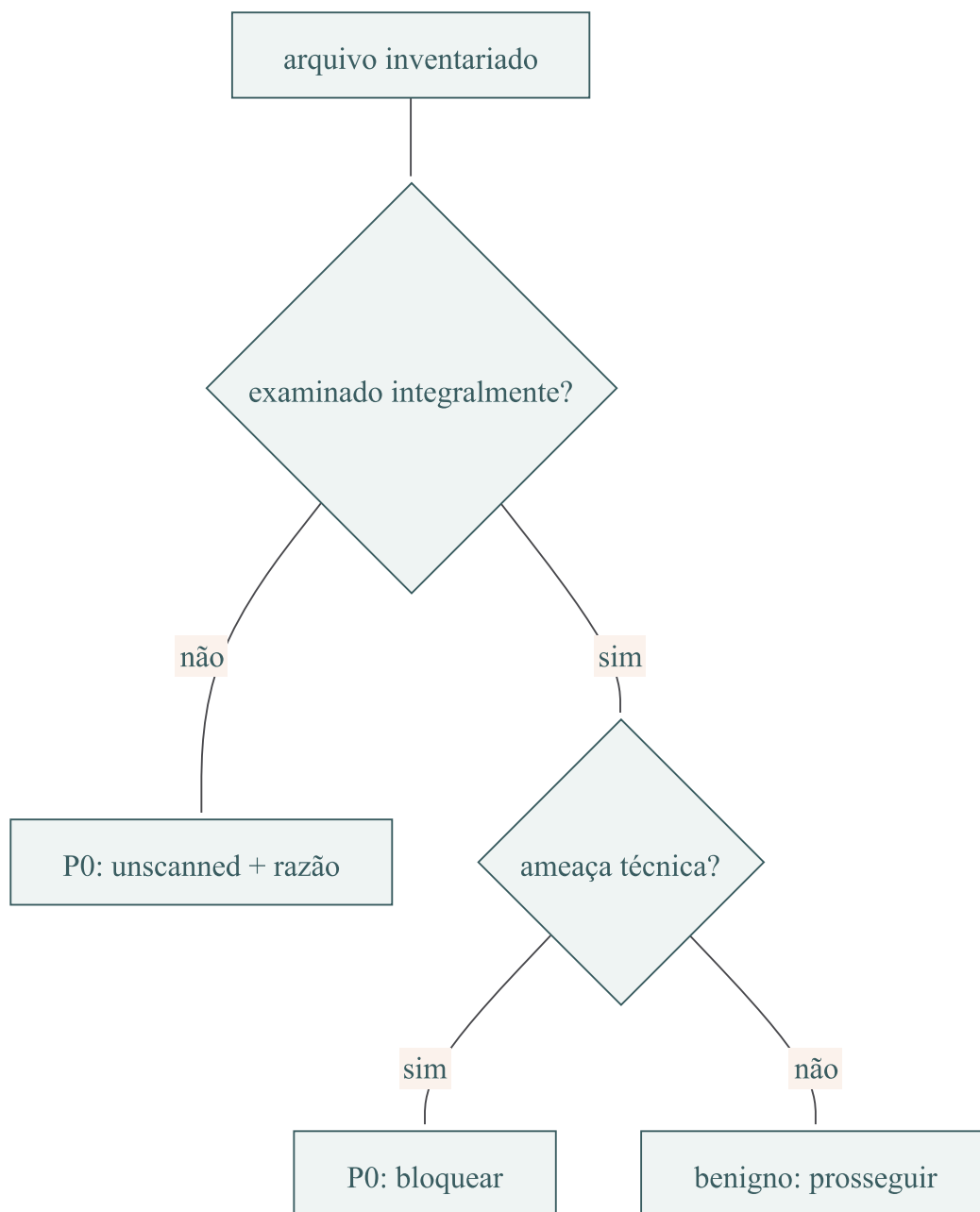
D10 — Injection scan fail-closed

Diagrama D10 — D10 — Injection scan fail-closed • largura final 14.2 cm

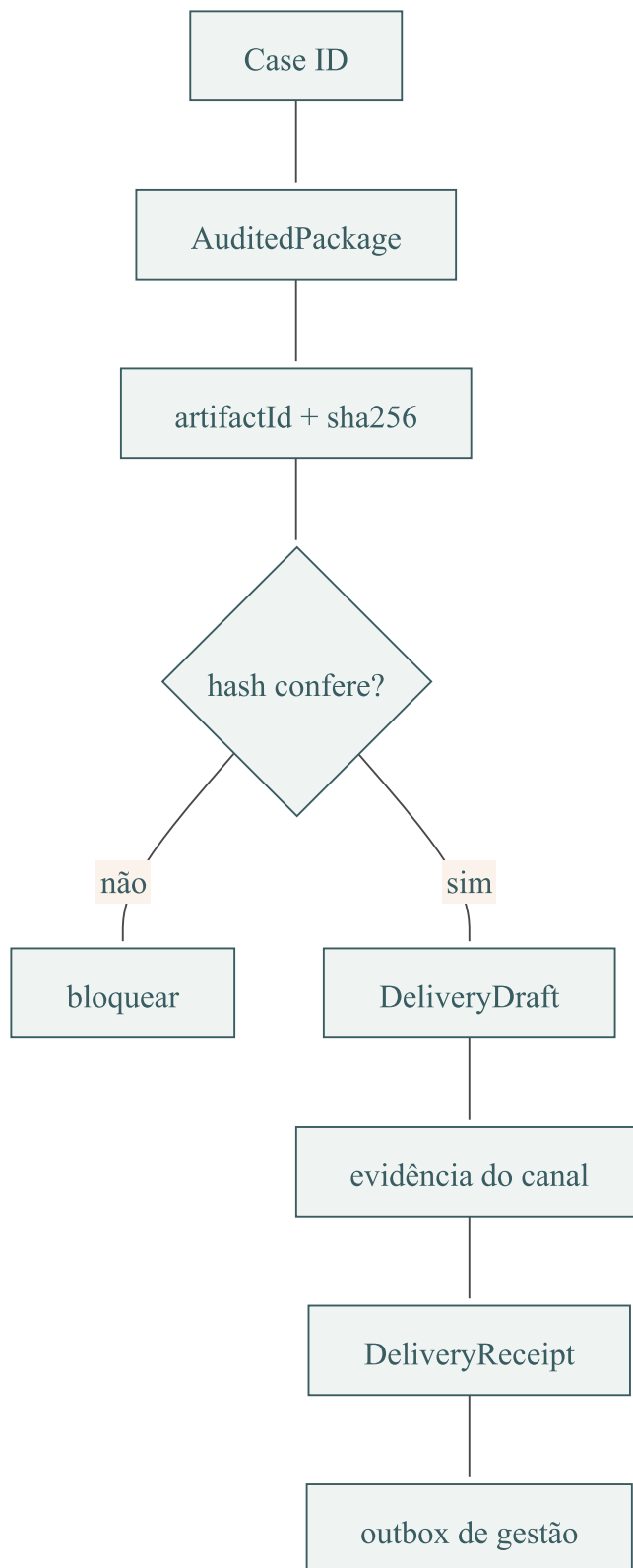
D11 — Entrega por identidade

Diagrama D11 — D11 — Entrega por identidade • largura final 8.9 cm

D12 — ManagementOutbox

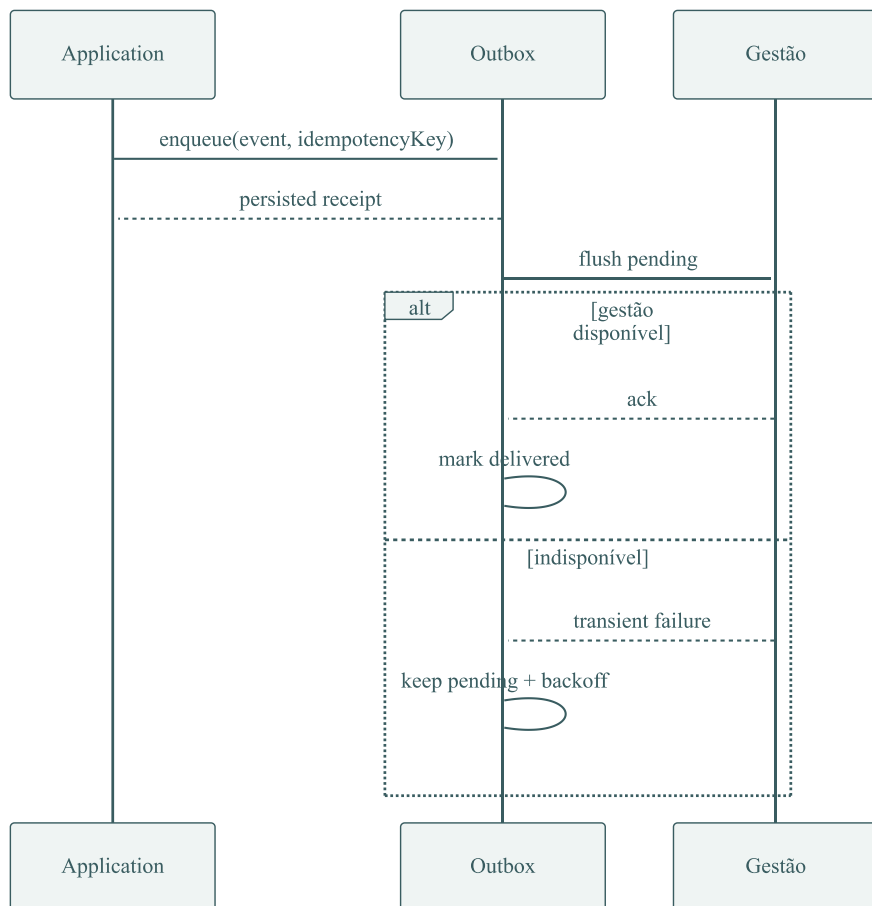


Diagrama D12 — D12 — ManagementOutbox • largura final 13.3 cm

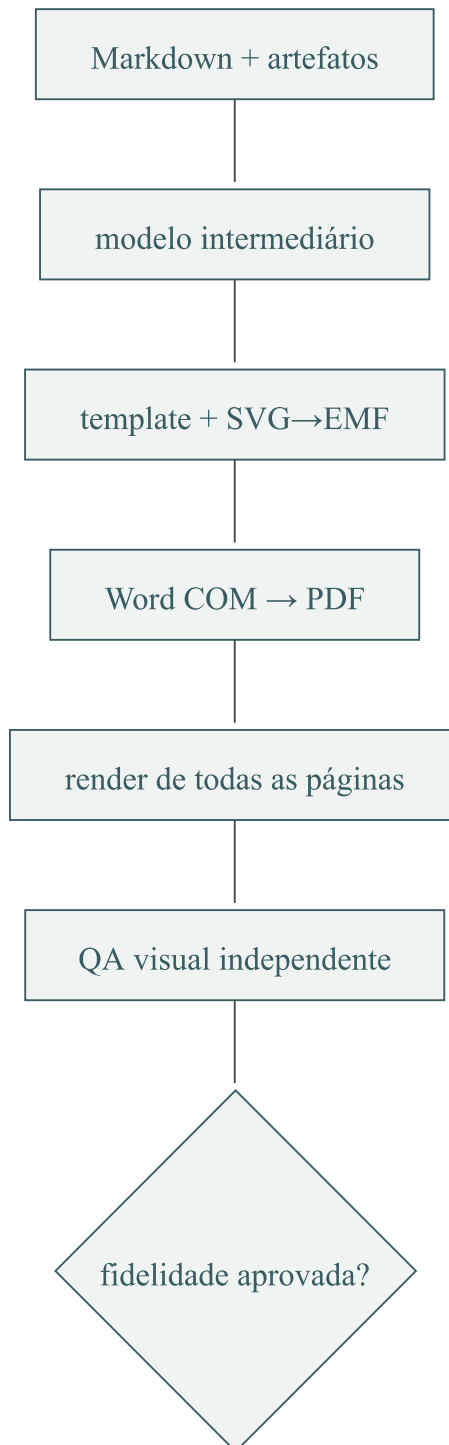
D13 — Renderização e QA

Diagrama D13 — D13 — Renderização e QA • largura final 6.4 cm

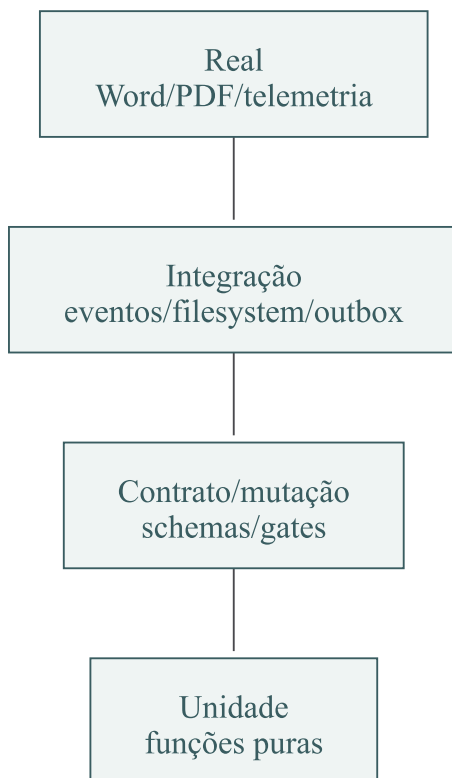
D14 — Pirâmide de testes

Diagrama D14 — D14 — Pirâmide de testes • largura final 6.4 cm

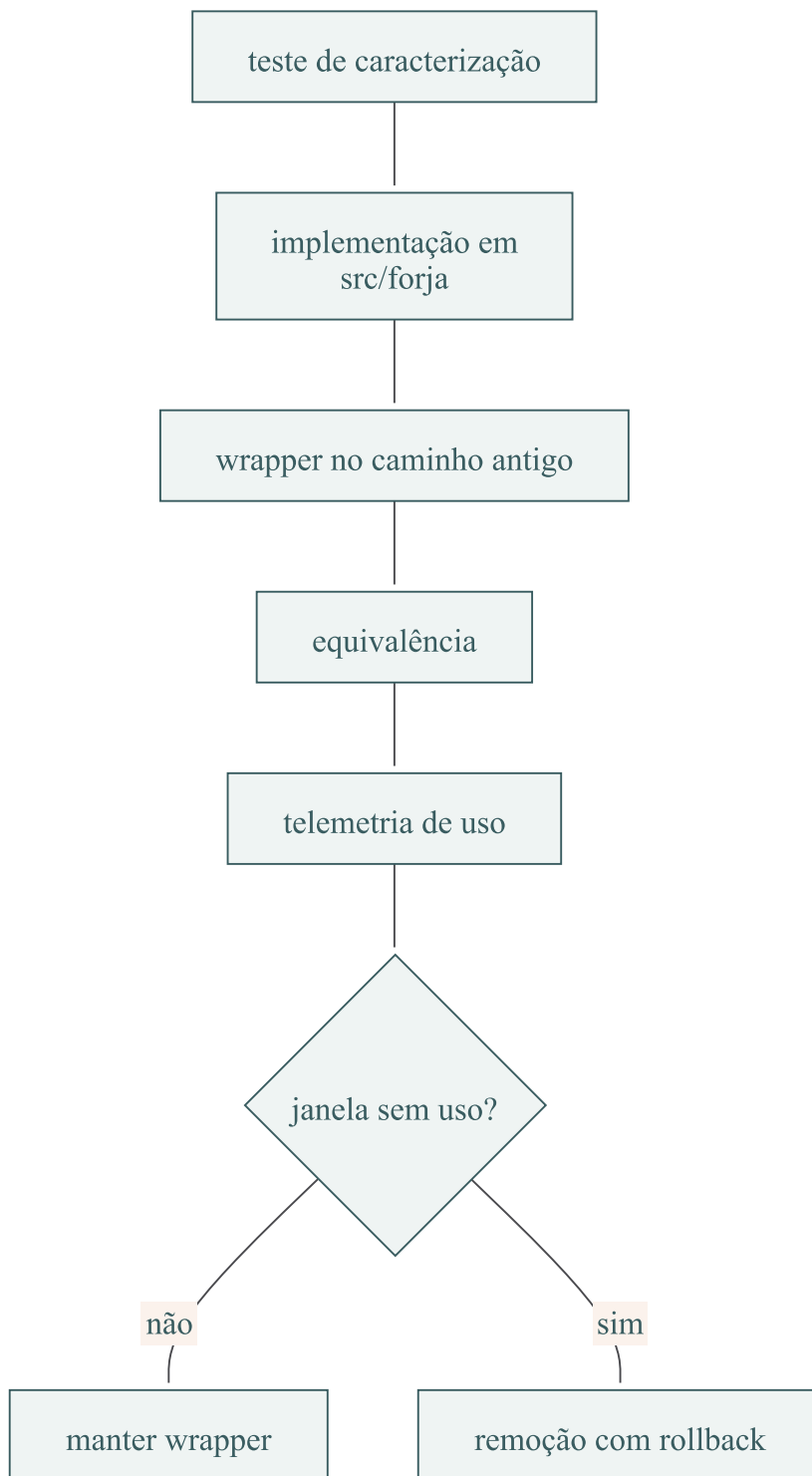
D18 — Migração física por fachada

Diagrama D18 — D18 — Migração física por fachada • largura final 11.2 cm

D19 — Ciclo de shim

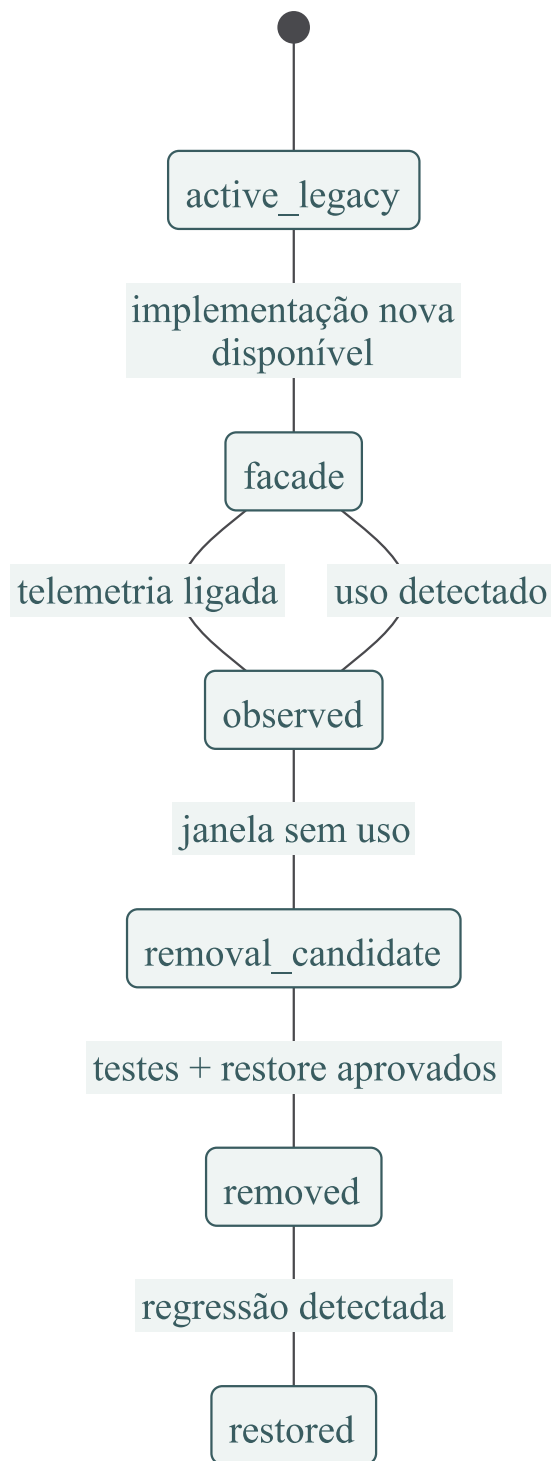


Diagrama D19 — D19 — Ciclo de shim • largura final 7.7 cm

3. Descoberta da régua

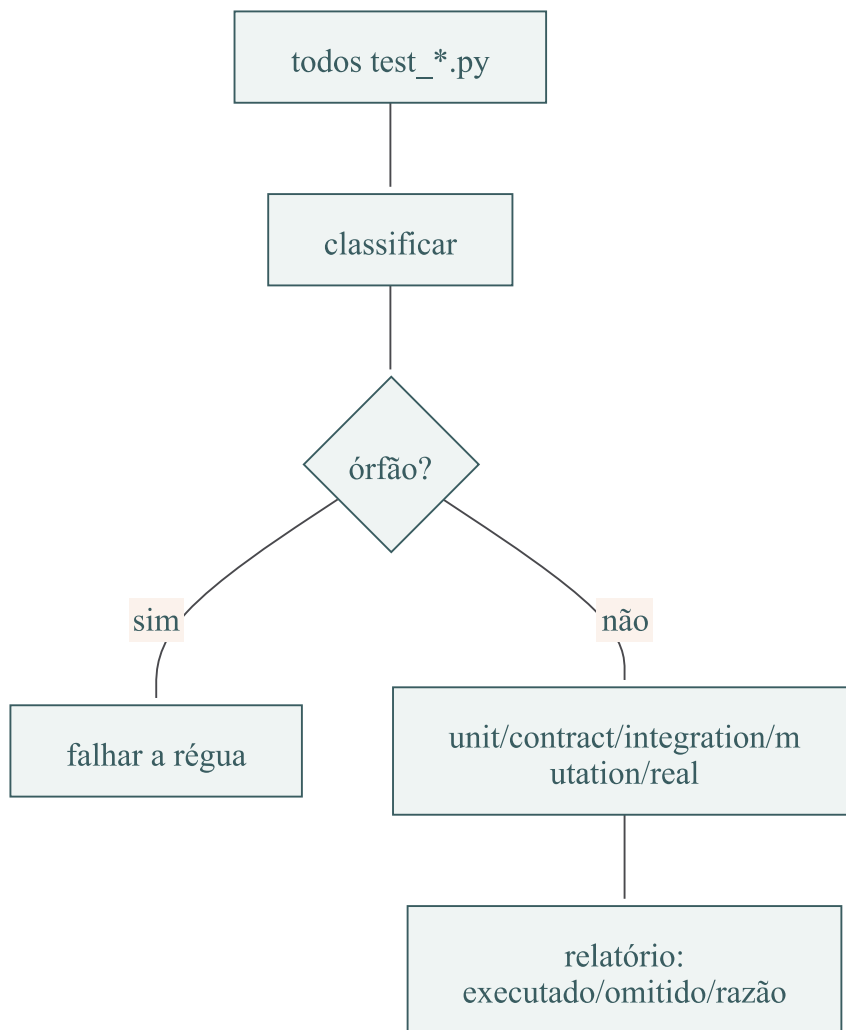
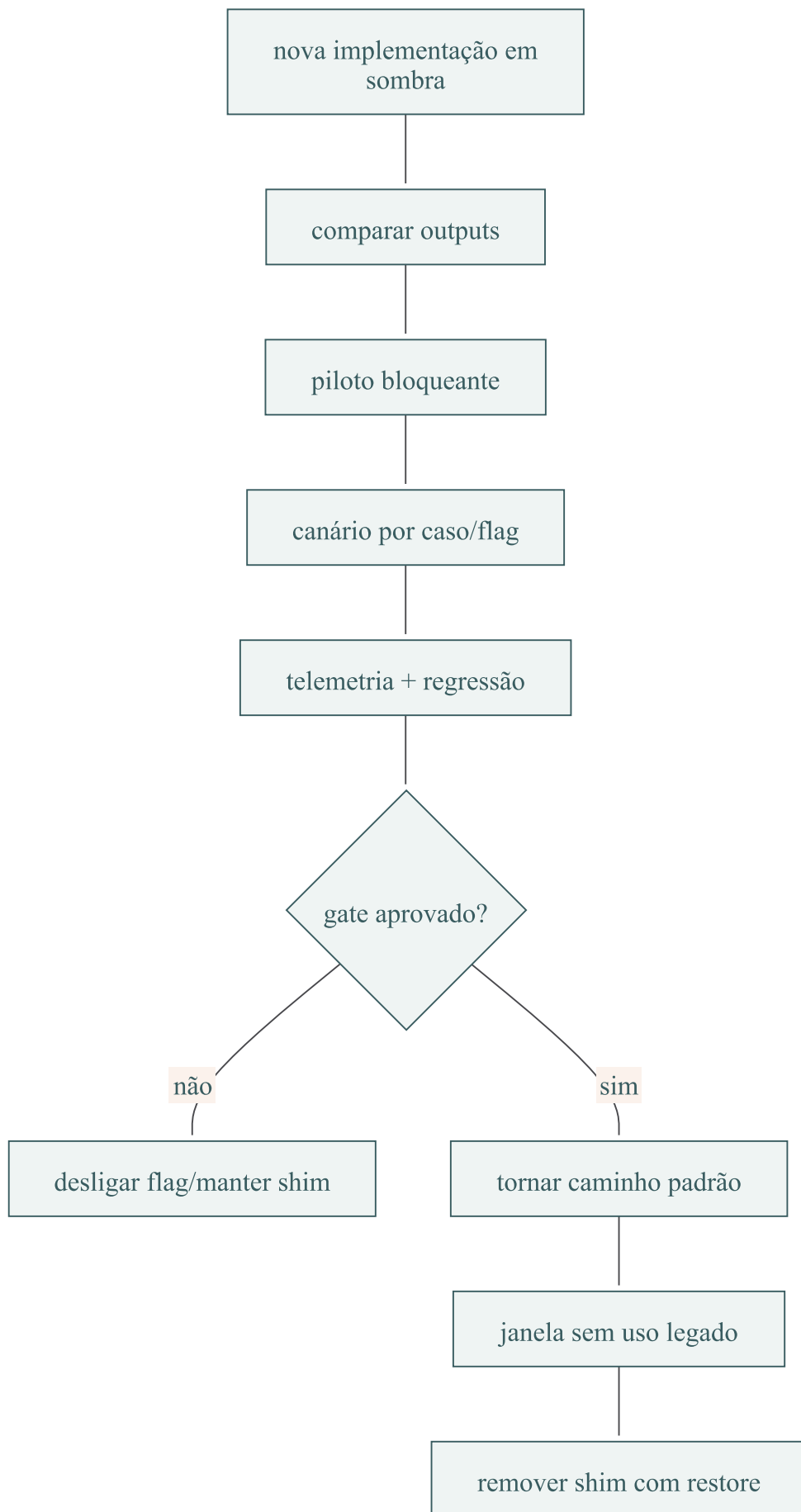


Diagrama S13 — 3. Descoberta da régua • largura final 11.6 cm

11. Cutover



10. Decisão de início

GATE DE AUTORIZAÇÃO

A execução começa somente quando este plano for aprovado como referência e o programa entrar em R0. A aprovação do documento não autoriza exclusões, cutover, promoção normativa ou comunicação externa.

Primeiro lote recomendado

- 55. P00 — baseline isolado, inventário e contratos observados.
- 56. P01 — descoberta canônica de toda a régua de testes.
- 57. P02–P04 — correção TDD dos quatro gates P0.
- 58. P05 — ambiente reproduzível e esqueleto modular sem mover lógica.

Conclusão do planejamento: o trabalho está decomposto, rastreável, reversível e pronto para execução por ondas. A próxima mudança funcional segura é iniciar R0, não mover arquivos nem deduplicar código diretamente.